Conceptos de Sistemas Operativos (IC)
Introducción a los Sistemas Operativos (LI, LS, APU, ATIC)

Trabajo Práctico N° 4

Objetivo

Continuar los temas relacionados a la administración de memoria, introduciendo paginación por demanda y su utilización para implementar memoria virtual, analizando el impacto en la ejecución de los procesos.

Analizar el componente de Entrada/Salida de un Kernel, su organización, objetivos y funcionalidades en la administración de los dispositivos. Estudiar la implementación de Sistemas de Archivos. Estudiar el impacto de la utilización de Cache de Disco.

Temas incluidos

Paginación por demanda. Algoritmos de reemplazos. Memoria Virtual. Entrada-Salida. Dispositivos. Administración de Discos. Sistemas de Archivos. Buffer Cache.

1. Memoria Virtual

a. Describa qué beneficios introduce este esquema de administración de la memoria.

La Memoria Virtual es un esquema de administración de memoria que separa la memoria lógica (la que ve el programador y el proceso) de la memoria física (la RAM disponible).

Los principales beneficios que introduce son:

- Mayor Grado de Multiprogramación: Permite que la suma de la memoria lógica de todos los procesos sea mayor que la memoria física disponible. El *kernel* solo carga las partes de los procesos que se están utilizando activamente, dejando el resto en el disco.
- Facilita la Programación: El programador ya no necesita preocuparse por la cantidad de memoria física disponible ni por la reubicación de direcciones. Cada proceso tiene su propio espacio de direcciones contiguo.
- Simplificación del Código: La memoria lógica de un proceso puede ser mayor que la memoria física total de la máquina.
- Reubicación Sencilla: El código puede cargarse en cualquier posición de la memoria física.
- Aislamiento y Protección: Los espacios de direcciones de los diferentes procesos están aislados entre sí, impidiendo que un proceso acceda o corrompa la memoria de otro proceso.
- Compartición de Memoria: Permite que varios procesos compartan páginas de

memoria de manera segura (ejemplo: bibliotecas compartidas).

b. ¿En que se debe apoyar el Kernel para su implementación?

El Kernel debe apoyarse en dos componentes clave para implementar la memoria virtual, particularmente usando paginación:

1. Hardware de la CPU (MMU - Memory Management Unit): La MMU es la unidad de gestión de memoria del hardware. Se encarga de la traducción de direcciones de forma rápida, convirtiendo la dirección virtual solicitada por el CPU en una dirección física real en la RAM. Para esto utiliza la Tabla de Páginas.
2. Dispositivo de Paginación (Swap Space): Es un área dedicada en el disco duro (o SSD) donde el Kernel almacena las páginas de los procesos que no caben o no se están usando activamente en la memoria principal.

c. Al implementar está técnica utilizando paginación por demanda, las tablas de páginas de un proceso deben contar con información adicional además del marco donde se encuentra la página. ¿Cuál es está información? ¿Por qué es necesaria?

La paginación por demanda es traer una página a la memoria principal (RAM) solo cuando es realmente necesaria, es decir, cuando un proceso intenta acceder a una dirección dentro de esa página.

Información Adicional	Propósito y Necesidad
Bit V (Válido/Inválido)	**Necesario para la Demanda:** Indica si la página virtual está o no en la memoria principal (RAM). Si este bit es 0 y la CPU intenta acceder a la página, se genera un **Fallo de Página** (*Page Fault*), lo que indica al *kernel* que debe traer la página del disco.
Bit R (Referencia)	**Necesario para el Reemplazo:** El hardware lo activa (R=1) cada vez que se accede a la página. Es vital para que el *kernel* pueda implementar algoritmos de reemplazo de páginas (como LRU o Segunda Chance) que se basan en saber qué páginas han sido usadas recientemente

Bit M (Modificación / Sucio)	**Necesario para la Escritura a Disco:** Es activado (M=1) por el hardware cuando se escribe sobre la página (modificación). Esto es crucial para el *kernel* porque, si la página va a ser reemplazada, solo necesita escribirla de vuelta al disco si M=1. Si M=0, la copia en disco es idéntica y se puede reemplazar directamente, ahorrando E/S.
Bits de Protección	**Necesario para la Seguridad:** Indican los permisos de acceso a la página (lectura, escritura, ejecución). Esto protege al sistema de accesos no autorizados a ciertas áreas de la memoria.

2. **Fallos de Página (Page Faults):**

a. ¿Cuándo se producen?

Un Fallo de Página se produce cuando un proceso intenta acceder a una dirección de memoria virtual cuya página correspondiente **no se encuentra cargada actualmente en la memoria principal (RAM)**.

Esto ocurre específicamente cuando:

- La Unidad de Gestión de Memoria (MMU) revisa la **Tabla de Páginas** del proceso.
- El **Bit Válido/Inválido (V)** de la entrada de la página es **0** (Inválido) , indicando que la página reside en el dispositivo de paginación (disco).

b. ¿Quién es responsable de detectar un fallo de página?

La responsabilidad de detectar un fallo de página recae en el Hardware , específicamente en la Unidad de Gestión de Memoria (MMU) de la CPU.

- El hardware es el único que puede verificar el Bit V (válido/inválido) en la tabla de páginas durante la traducción de la dirección.
- Al detectar que el Bit V es 0, el hardware dispara una interrupción (una trampa o *trap*) que transfiere el control al *kernel* del sistema operativo para que este pueda gestionar el fallo.

c. Describa las acciones que emprende el Kernel cuando se produce un fallo de página.

Cuando el *kernel* recibe la interrupción del fallo de página, inicia una secuencia de acciones para resolver la situación y reanudar el proceso.

Las acciones típicas que emprende el *kernel* son:

1. **Validar la Referencia:** El *kernel* primero determina si la referencia a la página

es **válida** o no (es decir, si la dirección virtual a la que se intentó acceder está realmente dentro del espacio de direcciones legal del proceso). Si no es válida, termina el proceso con un error (ej., violación de segmento).

2. **Buscar la Página en Disco:** Si la referencia es legal, el *kernel* localiza la página requerida en el **dispositivo de paginación** (área de *swap* en el disco).
3. **Seleccionar una Víctima:** El *kernel* debe encontrar un **marco libre** en la memoria principal para cargar la página nueva. Si no hay marcos libres, debe invocar un **Algoritmo de Reemplazo** (como LRU o FIFO) para seleccionar una página existente en RAM como **víctima**.
4. **Gestión del Bit Modificado (M):**
   - Si la página víctima seleccionada tiene el **Bit M (Modificado o *Dirty*) en 1** , el *kernel* debe escribir (descargar) esa página de vuelta al disco antes de reemplazarla.
   - Si el Bit M es **0**, la página se puede descartar directamente.
5. **Cargar la Nueva Página:** El *kernel* programa la operación de E/S para leer la página requerida desde el disco y cargarla en el marco libre recién obtenido. Durante este tiempo, el proceso que causó el fallo pasa al estado de espera.
6. **Actualizar la Tabla de Páginas:** Una vez que la página ha sido cargada en RAM, el *kernel* actualiza la entrada en la Tabla de Páginas:
   - Establece el **Bit V (Válido)** a **1**.
   - Establece el campo **Marco** con el número de marco físico donde se cargó la página.
   - Reinicia los bits R y M a 0.
7. **Reanudar el Proceso:** Finalmente, el *kernel* devuelve el control al CPU para que **reanude la instrucción** que originalmente causó el fallo de página, permitiendo que la traducción de la dirección se complete con éxito.

3. Suponga que la tabla de páginas para un proceso que se está ejecutando es la que se muestra a continuación:

Página	Bit V	Bit R	Bit M	Marco
0	1	1	0	4
1	1	1	1	7
2	0	0	0	-
3	1	0	0	2
4	0	0	0	-
5	1	0	1	0

Asumiendo que:

El tamaño de la página es de 512 bytes

Cada dirección de memoria referencia 1 byte
Los marcos se encuentran contiguos y en orden en memoria
(0, 1, 2..) a partir de la dirección física 0.

¿Qué dirección física, si existe, correspondería a cada una de las siguientes direcciones virtuales? (No gestionar ningún fallo de página en caso de producirse)

a. 1052
b. 2221
c. 5499
d. 3101

4**. Analice cómo impacta el tamaño de una página (pequeña o grande) en paginación por demanda.**

Tamaño de pagina pequeño:

Aspecto	Impacto	Justificación
Fragmentación Interna	**Baja**	Se reduce el espacio desperdiciado dentro de la última página asignada a un proceso, ya que la porción no utilizada es pequeña. Esto mejora la eficiencia de la RAM.
Tabla de Páginas	**Grande**	Para mapear el mismo espacio de direcciones virtual, se necesita un mayor número de páginas, lo que resulta en una tabla de páginas más grande.
Fallos de Página	**Alta**	Dado que cada página contiene menos datos, la **Localidad de Referencia** no se aprovecha tan bien. Si un proceso necesita acceder a datos que estaban justo después de la página que acaba de cargar, necesitará otra E/S inmediata, aumentando la tasa de fallos.
E/S y Rendimiento	**Lenta**	Más fallos de página implican más accesos al disco (E/S) y más interrupciones. Esto aumenta la sobrecarga del sistema (*overhead*) y reduce el rendimiento de la CPU.

Tamaño de pagina Grande:

Aspecto	Impacto	Justificación
Fragmentación Interna	**Alta**	El espacio desperdiciado dentro de la última página de un proceso (la parte no utilizada) puede ser grande. Esto reduce la cantidad de memoria utilizable.
Tabla de Páginas	**Pequeña**	Se requiere un menor número de entradas en la tabla de páginas para cubrir el mismo espacio de direcciones virtual. Esto reduce el consumo de memoria del *kernel* por las tablas.
Fallos de Página	**Baja**	Una página grande es más probable que contenga los datos y el código que el proceso necesitará en el futuro cercano (mejor **Localidad de Referencia**). Esto reduce la tasa de fallos de página.
E/S y Rendimiento	**Rápida**	Menos fallos de página y una E/S más eficiente (se transfieren bloques grandes de datos de una sola vez). Esto reduce la sobrecarga del *kernel* y mejora la utilización de la CPU.

5. A**signación de marcos a un proceso (Conjunto de trabajo o Working Set). Con la memoria virtual paginada, no se requiere que todas las páginas de un proceso se encuentren en memoria. El Kernel debe controlar cuantas páginas de un proceso puede tener en la memoria principal. Existen 2 políticas que se pueden utilizar:**

   Asignación Fija

   Asignación Dinámica.

   a. Describa cómo trabajan estas 2 políticas.

Las políticas de asignación de marcos determinan cuánta memoria física (cuántos marcos) se le deben dar a un proceso que se está ejecutando en un ambiente de multiprogramación con memoria virtual.

1. Asignación Fija

- **Descripción:** A cada proceso se le asigna un **número fijo de marcos** de memoria que permanece constante durante toda su ejecución.
- **Decisión:** Este número se determina al inicio del proceso y puede ser basado en el reparto equitativo o proporcional de los marcos disponibles en el sistema.
- **Implicación en el Reemplazo:** Cuando el proceso causa un Fallo de Página, debe usar un **Reemplazo Local**. Es decir, si necesita una nueva página, debe reemplazar una de las páginas que **actualmente le pertenecen** (dentro de su asignación fija). Su asignación no aumenta ni disminuye.

2. Asignación Dinámica

- **Descripción:** El número de marcos asignados a un proceso **puede variar** a lo largo del tiempo, aumentando o disminuyendo durante su ejecución.
- **Decisión:** La asignación se ajusta dinámicamente basándose en la necesidad real de memoria del proceso, a menudo utilizando técnicas como el **Conjunto de Trabajo (*Working Set*)** o la **Frecuencia de Fallos de Página (PFF)**.
- **Implicación en el Reemplazo:** Generalmente, está ligada a la política de **Reemplazo Global**. Cuando ocurre un Fallo de Página, el proceso puede tomar marcos prestados de otros procesos (aumentando su asignación) o reemplaza una de sus propias páginas (disminuyendo su asignación), buscando siempre optimizar el rendimiento global del sistema

b. Dada la siguiente tabla de procesos y las páginas que ellos ocupan, y teniéndose 40 marcos en la memoria principal, cuántos marcos le corresponderá a cada proceso si se usa la técnica de Asignación Fija: ○ **Reparto Equitativo**

○ **Reparto Proporcional**

Proceso	**Total de Páginas Requeridas**
1	15
2	20
3	20
4	8

c. ¿Cuál de los 2 repartos usados en b) resultó más eficiente? Justifique

Equitativo: Asigna 10 marcos a todos. Esto es **ineficiente** porque le da **demasiados** marcos al Proceso 4 (que solo necesita 8 páginas en total, desperdiciando 2 marcos) y **pocos** marcos a los Procesos 2 y 3 (que son los más

grandes y necesitan 20 páginas cada uno). Esto podría llevar a que P2 y P3 sufran una alta tasa de fallos de página (*thrashing*).
Proporcional: Asigna la memoria según la **demanda relativa** del proceso. Los procesos más grandes (P2 y P3) reciben una mayor porción de la memoria (13 y 12 marcos), lo que es justo y permite que sus conjuntos de trabajo sean mejor acomodados, reduciendo así la probabilidad de Fallos de Página y mejorando la eficiencia general del sistema.

6. **Reemplazo de páginas (selección de una víctima). ¿Qué sucede cuando todos los marcos en la memoria principal están usados por las páginas de los procesos y se produce un fallo de página? El Kernel debe seleccionar una de las páginas que se encuentra en memoria principal (en un marco) como víctima, la que será reemplazada por la nueva página que produjo el fallo.**

Considere los siguientes algoritmos de selección de víctimas básicos: LRU
FIFO
OPT (Óptimo)

Segunda Chance

a. Clasifique estos algoritmos de malo a bueno de acuerdo a la tasa de fallos de página que se obtienen al utilizarlos.

Ranking	Algoritmo	Tasa de Fallos de Página
1 (Peor)	FIFO (First In, First Out) 6	Alta
2	Segunda Chance (Second Chance) 7	Intermedia (Mejor que FIFO)
3	LRU (Least Recently Used) 8	Baja
4 (Óptimo)	OPT (Óptimo) 9	La más baja posible (Ideal Teórico)

b. Analice su funcionamiento. ¿Cómo se podrían implementar?

Algoritmo	Funcionamiento (Regla de Selección)	Posible Implementación
FIFO	Reemplaza la página que más tiempo ha estado en memoria (la más antigua)[11].	Se implementa con una cola (queue). La página que está al frente de la cola es la víctima.
LRU	Reemplaza la página que hace más tiempo que no ha sido referenciada (la menos recientemente usada)[12].	Se implementa idealmente con una lista enlazada donde cada referencia mueve la página al frente. Es costoso de mantener en hardware; a menudo se usan aproximaciones (como el uso del Bit R).
OPT (Óptimo)	Reemplaza la página que no se usará durante el período de tiempo más largo en el futuro[13].	No implementable. Se utiliza como *benchmark* (estándar de oro).
Segunda Chance	Es una mejora de FIFO que usa el Bit R (Referencia). Reemplaza la página más antigua (FIFO), *solo si* su Bit R es 0. Si el Bit R es 1, le da una "segunda oportunidad" (pone R=0 y la mueve al final de la cola, como si acabara de llegar)[14].	Se implementa con una cola circular y el Bit R del hardware.

c. Sabemos que la página a ser reemplazada puede estar modificada. ¿Cómo detecta el Kernel esta situación? ¿Qué acciones adicionales deben realizarse cuando se encuentra ante esta situación?

El Kernel detecta si una página ha sido modificada utilizando el Bit M (Modificado / Sucio), que se encuentra en la entrada de la página en la Tabla de Páginas.

- El Bit M es configurado a 1 por el hardware (MMU) cada vez que se realiza una operación de escritura sobre la página.

Acciones Adicionales (Escritura de Retorno)

Cuando el Kernel detecta que la página víctima está modificada (`M=1`), debe realizar una acción adicional crítica antes de cargar la nueva página:

- Escribir de Vuelta al Disco (*Write Back*): El Kernel debe programar una operación de E/S (Entrada/Salida) para escribir la página modificada de vuelta al dispositivo de paginación (disco). Esto asegura que los cambios realizados por el proceso no se pierdan.
- Aumenta la Latencia: Esta operación de escritura adicional aumenta significativamente el tiempo total de atención al Fallo de Página, ya que requiere dos operaciones de disco (una para escribir la víctima y otra para leer la página solicitada) en lugar de solo una.

7. Alcance del reemplazo. Al momento de tener que seleccionar una página victima, el Kernel puede optar por 2 políticas a utilizar:

Reemplazo local

Reemplazo global

a. Describa cómo trabajan estas 2 políticas.

1. Reemplazo Local

- Descripción: Un proceso solo puede seleccionar una página víctima para reemplazar dentro de su propia asignación de marcos de memoria.
- Funcionamiento: Si el Proceso A sufre un fallo de página, buscará un marco libre solo entre las páginas que actualmente pertenecen al Proceso A.
- Impacto: El número de marcos asignados a un proceso permanece fijo durante su ejecución. Es decir, la asignación de marcos de los demás procesos está protegida, y el rendimiento de un proceso no afecta directamente al rendimiento de los demás.
- Asociación: Esta política es típicamente usada junto con la Asignación Fija de marcos.

2. Reemplazo Global

- Descripción: Un proceso puede seleccionar una página víctima para reemplazar de cualquier marco disponible en la memoria principal, incluso si ese marco pertenece a otro proceso.
- Funcionamiento: Si el Proceso A sufre un fallo, puede tomar un marco que actualmente esté ocupado por una página del Proceso B o C. El proceso que sufre el fallo "roba" el marco de la asignación de otro.
- Impacto: El número de marcos asignados a un proceso varía dinámicamente con el tiempo. Un proceso con alta tasa de fallos de página puede crecer a costa de procesos menos activos.
- Asociación: Esta política es típicamente usada junto con la Asignación Dinámica de marcos.

b. ¿Es posible utilizar la política de "Asignación Fija" de marcos junto con la política de "Reemplazo Global"? Justifique.

No, no es posible utilizarlas juntas de manera lógica y eficiente.

Justificación:

1. Conflicto de Definiciones:
   - La Asignación Fija define que a un proceso se le garantiza un número constante y fijo de marcos durante toda su vida útil.
   - El Reemplazo Global permite que un proceso seleccione una página víctima de *cualquier* marco, incluso si ese marco pertenece a otro proceso.
2. Violación de la Fijeza: Si la asignación de marcos del Proceso A es fija (ej., 10 marcos), pero el Proceso B utiliza el Reemplazo Global y selecciona uno de los marcos de A como víctima, la asignación de A se reduce a 9 marcos.
3. Conclusión: El acto de permitir el Reemplazo Global viola inmediatamente la regla de Asignación Fija, ya que el tamaño del conjunto de marcos de un proceso ya no sería constante. Por lo tanto, el Reemplazo Local es la única política de reemplazo compatible con la Asignación Fija.

8. Considere la siguiente secuencia de referencias de páginas:
1, 2, 15, 4, 6, 2, 1, 5, 6, 10, 4, 6, 7, 9, 1, 6, 12, 11, 12, 2, 3, 1, 8, 1, 13, 14, 15, 3, 8

a. Si se disponen 5 marcos. ¿Cuántos fallos de página se producirán si se utilizan las siguientes técnicas de selección de víctima? (Considere una política de Asignación Dinámica y Reemplazo Global)

i. Segunda Chance

1	2	15	4	6	2	1	5	6	10	4	6	7	9	1	6	12	11	12	2	3	1	8	1	12	14	15	3	8
1	1	1	1	1	1	1*	1	1	10	10	10	10	10	10	10	12	12	12*	12*	12*	12*	12	12	12*	12	12	12	12
	2	2	2	2	2*	2*	2	2	2	2	2	7	7	7	7	7	7	7	2	2	2	2	2	2	14	14	14	14
		15	15	15	15	15	5	5	5	5	5	5	9	9	9	9	9	9	9	3	3	3	3	3	3	3	3*	3*
			4	4	4	4	4	4	4	4*	4*	4*	4	1	1	1	1	1	1	1	1*	1	1*	1*	1	15	15	15
				6	6	6	6	6*	6*	6*	6*	6*	6	6	6*	6	11	11	11	11	11	8	8	8	8	8	8	8*

Cantidad de page faults=17

ii. FIFO

1	2	15	4	6	2	1	5	6	10	4	6	7	9	1	6	12	11	12	2	3	1	8	1	12	14	15	3	8
1	1	1	1	1	1	1	5	5	5	5	5	5	5	5	6	6	6	6	6	6	1	1	1	1	1	1	1	1
	2	2	2	2	2	2	2	2	10	10	10	10	10	10	10	12	12	12	12	12	12	8	8	8	8	8	8	8
		15	15	15	15	15	15	15	15	15	15	7	7	7	7	7	11	11	11	11	11	11	11	12	12	12	12	12
			4	4	4	4	4	4	4	4	4	4	9	9	9	9	9	9	2	2	2	2	2	2	14	15	15	15
				6	6	6	6	6	6	6	6	6	6	1	1	1	1	1	1	3	3	3	3	3	3	3	3	3

Page faults=20

iii. LRU

Con formato: Sangría: Izquierda: 0 cm

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
3	2	15	4	6	2	1	5	6	10	4	6	7	9	1	6	12	11	12	2	3	1	8	1	12	14	15	3	8
1[1]		5	1	5	1	1	5	5	1	5	5 [7]13		7	7	7	7 12[18]		12	12	12	11	11 8[23]		8	8	8	8	8 [28]	3
	2[2]		2	2	2 [28]		2	2	2	2 4[11]		4	4	4	4	4 12[17]		12	12 [19]		12	12	12	12	12	12 [25]		12	12	12	12
		15[3]		15	15	15	15 [28]	5	5	5	5	5 [14]		9	9	9	9	9 [23]		2	2	2	2	2	14[26]		14	14	14
			4[4]		4	4	4	4	4 10[13]		10	10	10		1	1	1	1	1 [21]		3	3	3	3		3	15[27]		15	15
				6[5]		6	6	6 [28]		6	6 [12]		6	6	6 [18]		6	6	6	6	6 [22]		1	1 [24]		1	1	1	1 [28]	

Page faults=20

iv. OPT

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
3	2	15	4	6	2	1	5	6	10	4	6	7	9	1	6	12	11	12	2	3	1	8	1	12	14	15	3	8
1[7]	1[7]	1[7]	1[7]	1[7]	1[7]	1[15]	5[*]	5[*]	10[*]	10[*]	10[*]	7[*]	9[*]	9[*]	12[18]	12[18]	12[25]	12[25]	12[25]	12[25]	12[25]	12[25]	12[*]	12[*]	12[*]	12[*]	12[*]	
	2[8]	2[8]	2[8]	2[8]	2[28]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[8]	2[*]	3[28]	3[28]	3[28]	3[28]	3[28]	3[28]	3[28]	3[*]	3[28]
		15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[27]	15[*]	15[*]	15[*]
			4[11]	4[11]	4[11]	4[11]	4[11]	4[11]	4[11]	4[*]	4[*]	7[*]	7[*]	3[22]	1[22]	3[22]	3[22]	3[22]	3[22]	3[22]	3[24]	3[24]	1[*]	1[*]	14[*]	14[*]	14[*]	14[*]
				6[6]		6[6]	6[6]	6[12]	6[12]	6[12]	6[18]	6[18]	6[18]	6[18]	6[*]	6[*]	14[*]	14[*]	14[*]	14[*]	14[*]	8[28]	8[28]	8[28]	8[28]	8[28]	8[28]	6[*]

Page faults=15

b. Suponiendo que cada atención de un fallo se página requiere de 0,1 seg. Calcular el tiempo consumido por atención a los fallos de páginas para los algoritmos de a).

Algoritmo	Fallos de Página (FP)	Cálculo (FP×0.1 seg)	Tiempo Total Consumido
Segunda Chance	17	17×0.1	1.7 segundos
FIFO	20	20×0.1	2.0 segundos
LRU	20	20×0.1	2.0 segundos
OPT (Óptimo)	15	15×0.1	1.5 segundos

9. Sean los procesos A, B y C tales que necesitan para su ejecución las siguientes páginas:

A: 1, 3, 1, 2, 4, 1, 5, 1, 4, 7, 9, 4

B: 2, 4, 6, 2, 4, 1, 8, 3, 1, 8

C: 1, 2, 4, 8, 6, 1, 4, 1

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguientes secuencia:

1. B demanda 2 páginas 2. A demanda 3 páginas 3. C demanda 2 páginas 4. B demanda 3 páginas 5. A demanda 3 páginas 6. C demanda 2 páginas 7. B demanda 2 páginas 8. C demanda 4 páginas 9. A demanda 3 páginas 10. B demanda 3 páginas 11. C termina 12. A demanda 3 páginas 13. B termina 14. A termina

a. Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctimas:

LRU

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
Proceso	**B**	**B**	**A**	**A**	**A**	**C**	**C**	**B**	**B**	**B**	**A**	**A**	**A**	**C**	**C**	**B**	**B**	**C**	**C**	**C**	**C**	**A**	**A**	**A**	**B**	**B**	**B**	**F(C)**	**A**	**A**	**A**	**F(B)**	**F(A)**
marcos/paginas	2	4	1	3	1	1	2	6	2	4	2	4	1	4	8	1	8	6	1	4	1	5	1	4	3	1	8		7	9	4		
1	2[1]	2[1]	2[1]	2[1]	2[1]	2[1]	2[7]	2[7]	2[9]	2[9]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	2[11]	9[30]	9[30]	9[30]	
2		4[2]	4[2]	4[2]	4[2]	4[2]	4[2]	4[2]	4[2]	4[10]	4[10]	4[12]	4[12]	4[14]	4[14]	4[14]	4[14]	4[14]	4[14]	4[20]	4[20]	4[20]	4[20]	4[24]	4[24]	4[24]	4[24]	4[24]	4[24]	4[24]	4[31]	4[31]	
3			1[3]	1[3]	1[5]	1[6]	1[6]	1[6]	1[6]	1[6]	1[6]	1[6]	1[13]	1[13]	1[13]	1[16]	1[16]	1[16]	1[19]	1[19]	1[21]	1[21]	1[23]	1[23]	1[23]	1[26]	1[26]	1[26]	1[26]	1[26]	1[26]		
4				3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[4]	3[25]	3[25]	3[25]	3[25]	3[25]	3[25]	3[25]		
5								6[8]	6[8]	6[8]	6[8]	6[8]	6[8]	6[8]	6[8]	6[8]	6[8]	6[18]	6[18]	6[18]	6[18]	6[18]	6[18]	6[18]	6[18]	6[18]	6[18]		7[29]	7[29]	7[29]	7[29]		
6															8[15]	8[15]	8[17]	8[17]	8[17]	8[17]	8[17]	8[17]	8[17]	8[17]	8[17]	8[27]	8[27]	8[27]	8[27]	8[27]	8[27]		
7																						5[22]	5[22]	5[22]	5[22]	5[22]	5[22]	5[22]	5[22]	5[22]	5[22]		
Faut page= 9																																	

ii. Segunda Chance

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
Proceso	**B**	**B**	**A**	**A**	**A**	**C**	**C**	**B**	**B**	**B**	**A**	**A**	**A**	**C**	**C**	**B**	**B**	**C**	**C**	**C**	**C**	**A**	**A**	**A**	**B**	**B**	**B**	**F(C**	**A**	**A**	**A**	**F(B**	**F(A)**
marcos/paginas	2	4	1	3	1	1	2	6	2	4	2	4	1	4	8	1	8	6	1	4	1	5	1	4	3	1	8		7	9	4		
1	2	2	2	2	2	2	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2*	2	2	2	
2		4	4	4	4	4	4	4	4	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4*	4	4*	4*	
3			1	1	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1*	1	1		
4				3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3*	3*	3*	3*	3*	3	3		
5								6	6	6	6	6	6	6	6	6	6	6	6*	6*	6*	6*	6*	6*	6*	6*	6*		7	9	9	9	
6																8	8	8*	8*	8*	8*	8*	8*	8*	8*	8*	8*	8*	8*	8*	8*		
7																							5	5	5	5	5	5	5	5	5	5	
Fault page= 9																																	

b. Considerando una política de Asignación Fija con reparto equitativo y Reemplazo Local y disponiéndose de 9 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctimas:

i. LRU

LRU	**Proceso A**	1	2	3	4	5	6	7	8	9	10	11	12
	marco/paginas	1	3	1	2	4	1	5	1	4	7	9	4
	1	1[1]	1[1]	1[3]	1[3]	1[3]	1[6]	1[6]	1[8]	1[8]	1[8]	9[11]	9[11]
	2		3[2]	3[2]	3[2]	4[5]	4[5]	4[5]	4[5]	4[9]	4[9]	4[9]	4[12]
	3				2[4]	2[4]	2[4]	5[7]	5[7]	5[7]	7[10]	7[10]	7[10]
	page faults:7												

LRU	**Proceso B**	1	2	3	4	5	6	7	8	9	10
	marco/paginas	2	4	6	2	4	1	8	3	1	8
	1	2[1]	2[1]	2[1]	2[4]	2[4]	2[4]	8[7]	8[7]	8[7]	8[10]
	2		4[2]	4[2]	4[2]	4[5]	4[5]	4[5]	3[8]	3[8]	3[8]
	3			6[3]	6[3]	6[3]	1[6]	1[6]	1[6]	1[9]	1[9]
	page faults: 6										

LRU	**Proceso C**	1	2	3	4	5	6	7	8
	marco/paginas	1	2	4	8	6	1	4	1
	1	1[1]	1[1]	1[1]	8[4]	8[4]	8[4]	4[7]	4[7]
	2		2[2]	2[2]	2[2]	6[5]	6[5]	6[5]	6[5]
	3			4[3]	4[3]	4[3]	1[6]	1[6]	1[8]
	page faults: 7								

Page faults total= 20

iii. Segunda Chance

segunda chance	**Proceso A**	1	2	3	4	5	6	7	8	9	10	11	12
	marco/paginas	1	3	1	2	4	1	5	1	4	7	9	4
	1	1	1	1*	1*	1	1*	1	1*	1*	1	9	9
	2		3	3	3	4	4	4	4	4*	4	4	4*
	3				2	2	2	5	5	5	7	7	7
	page faults: 7												

segunda chance	**Proceso B**	1	2	3	4	5	6	7	8	9	10
	marco/paginas	2	4	6	2	4	1	8	3	1	8
	1	2	2	2	2*	2*	2	8	8	8	8*
	2		4	4	4	4*	4	4	3	3	3
	3			6	6	6	1	1	1	1*	1*
	page faults: 6										

segunda chance	**Proceso C**	1	2	3	4	5	6	7	8
	marco/paginas	1	2	4	8	6	1	4	1
	1	1	1	1	8	8	8	4	4
	2		2	2	2	6	6	6	6
	3			4	4	4	1	1	1*
	page faults: 7								

10. Sean los procesos A, B y C tales que necesitan para su ejecución las siguientes páginas: **A**: 1, 2, 1, 7, 2, 7, 3, 2
B: 1, 2, 5, 2, 1, 4, 5
C: 1, 3, 5, 1, 4, 2, 3

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguiente manera:

1. **C** demanda 1 página
2. **A** demanda 2 páginas
3. **C** demanda 1 página 4. **B** demanda 1 página 5. **A** demanda 1 página
6. **C** modifica la página 1
7. **B** demanda 2 páginas
8. **A** demanda 1 página 9. **C** demanda 1 página 10. **B** modifica la página 2 11. **A** modifica la página 2
12. **B** demanda 2 páginas 13. **A** demanda 1 página 14. **B** demanda 2 páginas 15. **C** demanda 2 páginas
16. **C** demanda 1 página
17. **A** demanda 1 página
18. **B** termina
19. **A** demanda 2

páginas 20. **C** demanda 1 página 21. **A** termina 22. **C** termina

Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos, debiéndose guardar 1 marco para la gestión de descarga asincrónica de páginas modificadas ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctima:
a. Segunda Chance
b. FIFO
c. LRU

11. Hiperpaginación (Trashing)
a. ¿Qué es?

La hiperpaginación (o *Thrashing*) es un estado crítico en el que el sistema operativo pasa más tiempo paginando (intercambiando páginas entre la memoria RAM y el disco) que ejecutando las instrucciones reales de los procesos.

- Síntoma principal: El rendimiento del sistema colapsa drásticamente.
- Lo que ocurre: Los procesos no tienen suficientes marcos para contener las páginas que usan frecuentemente. Apenas cargan una página, se produce un fallo inmediato que obliga a sacar otra página que también se necesita, creando un ciclo vicioso de fallos de página constantes

b. ¿Cuáles pueden ser los motivos que la causan?

La causa raíz suele ser que la suma de la memoria que necesitan los procesos activos supera la memoria física total disponible. Los motivos específicos incluyen:

1. **Grado de multiprogramación demasiado alto:** Hay demasiados procesos ejecutándose a la vez para la cantidad de RAM disponible.
2. **Política de Reemplazo Global:** Si el algoritmo de reemplazo permite que un proceso "robe" marcos a otro proceso, y ese segundo proceso necesita esos marcos, empezará a fallar también, robándole a un tercero o al primero, generando una reacción en cadena.
3. **Tamaño del "Working Set" insatisfecho:** Si no se le garantiza a un proceso el mínimo de marcos necesarios para su *conjunto de trabajo* (las páginas que usa más frecuentemente en un momento dado), entrará en thrashing

c. ¿Cómo la detecta el SO?

El sistema operativo puede detectar que está entrando en hiperpaginación monitoreando dos métricas clave simultáneamente:

- Uso de CPU bajo: Como los procesos están todo el tiempo esperando que el disco traiga páginas, la CPU se queda sin trabajo ("idle").
- Uso del Dispositivo de Paginación (Disco) muy alto: El disco está saturado leyendo y escribiendo páginas constantemente.

La trampa del Planificador: Normalmente, si el planificador de CPU ve que el uso de CPU es bajo, piensa que "sobra capacidad" y carga *más* procesos. En situación de Thrashing, esto es fatal porque empeora el problema. Por eso es vital detectar la alta actividad de disco al mismo tiempo.

d. Una vez que lo detecta, ¿qué acciones puede tomar el Kernel para eliminar este problema?

Para salir de la hiperpaginación, el Kernel debe reducir la presión sobre la memoria.

Las acciones principales son:

1. Disminuir el grado de multiprogramación:

Es la solución más efectiva. El Kernel debe seleccionar uno o varios procesos (víctimas), suspenderlos y enviar todas sus páginas al disco (swap out completo).

2. Liberar marcos:

Al suspender esos procesos, sus marcos quedan libres y se pueden repartir entre los procesos sobrevivientes para que estos alcancen su tamaño de Working Set y dejen de fallar.

3. Implementar Control de Frecuencia de Fallos de Página (PFF):

El Kernel puede monitorear qué tan seguido falla cada proceso.

- Si la tasa es muy alta Le da más marcos.
- Si la tasa es muy baja Le quita marcos.
- Si no hay marcos libres para dar a quien falla mucho Se debe suspender un proceso.

12. Analice y compare las técnicas de PFF (Page Fault Frequency) y del Working

Set. ¿Como las Página 4 de 13

mismas permiten determinar la existencia de trashing?

Ambas técnicas buscan resolver el mismo problema: ¿Cuántos marcos debe tener asignado cada proceso para funcionar bien sin desperdiciar memoria? Sin embargo, lo hacen de formas muy distintas.

1. Modelo del Conjunto de Trabajo (Working Set)

Se basa en el principio de localidad. Asume que un proceso usa un subconjunto específico de páginas durante un periodo de tiempo.

- Cómo funciona: Se define una ventana de tiempo Δ (delta). El sistema mira hacia atrás Δ referencias de memoria. Las páginas únicas que fueron tocadas en ese lapso conforman el "Working Set" actual del proceso.

- Asignación: El S.O. intenta darle al proceso tantos marcos como tamaño tenga su

Working Set en ese momento.

2. Frecuencia de Fallos de Página (PFF)

Es un enfoque más reactivo y directo. En lugar de mirar qué páginas se usaron, mira "qué tan mal lo está pasando el proceso".

- Cómo funciona: Se establecen dos umbrales: un límite superior y un límite inferior de tasa de fallos.
    - Si la tasa de fallos supera el límite superior $\rightarrow$ El proceso necesita más memoria (se le da un marco).
    - Si la tasa de fallos cae por debajo del límite inferior $\rightarrow$ El proceso tiene demasiada memoria (se le quita un marco).

Característica	Working Set (WS)	PFF (Page Fault Frequency)
Enfoque	Preventivo: Intenta predecir lo que el proceso necesita basándose en la localidad reciente.	Reactivo: Reacciona cuando el proceso ya está sufriendo fallos o está demasiado holgado.
Costo	Alto: Es costoso para el hardware/S.O. rastrear cada referencia a memoria para mantener la ventana Δ actualizada.	Bajo: Es fácil de implementar, solo requiere contar las interrupciones por fallo de página.
¿Cómo detecta Trashing?	Sumando los tamaños de los WS de todos los procesos activos (D =Suma total de workigns sets	Cuando la tasa de fallos de los procesos es alta (supera el límite superior) y ya no quedan marcos libres en el sistema para asignarles.

Característica	Working Set (WS)	PFF (Page Fault Frequency)
	). Si > Total de Marcos Reales, entonces hay Trashing (la demanda supera la oferta física).	
Acción ante Trashing	Suspende un proceso entero para liberar sus marcos y que la suma baje a un nivel aceptable.	Suspende un proceso para liberar marcos y poder bajar la frecuencia de fallos de los restantes.

13. Considere un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda que utiliza un dispositivo de paginación, algoritmo de reemplazo global LRU y una política de asignación que reparte marcos equitativamente entre los procesos. El nivel de multiprogramación es actualmente, de 4.

Ante las siguientes mediciones:

a) Uso de CPU del 13%, uso del dispositivo de paginación del 97%.

b) Uso de CPU del 87%, uso del dispositivo de paginación del 3%.

c) Uso de CPU del 13%, uso del dispositivo de paginación del 3%.

Analizar para cada caso:

¿Qué sucede?

¿Puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU?

¿La paginación está siendo útil para mejorar el rendimiento del sistema?

a) CPU: 13% |
Dispositivo de Paginación: 97%

- ¿Qué sucede?: El sistema está en Hiperpaginación (Thrashing). El uso de la CPU es bajísimo porque los procesos están todos "bloqueados" esperando que el disco traiga páginas. El disco está al 97% saturado de trabajo intercambiando páginas. La memoria principal no alcanza para mantener el conjunto de trabajo (Working Set) de los 4 procesos activos.
- ¿Puede incrementarse el nivel de multiprogramación? NO, absolutamente no. Si agregas más procesos, robarán marcos a los procesos actuales (que ya están sufriendo), aumentando los fallos de página y empeorando el colapso. La acción correcta aquí es bajar el nivel de multiprogramación (suspender procesos).
- ¿La paginación es útil? En este momento no, está perjudicando el rendimiento severamente debido a la saturación**.**

b) CPU: 87% |

Dispositivo de Paginación: 3%

- ¿Qué sucede?: El sistema está funcionando de manera eficiente y saludable. La CPU está muy ocupada procesando instrucciones (87%), y hay muy pocos fallos de página (Disco al 3%), lo que significa que los procesos tienen cargadas en RAM las páginas que necesitan.
- ¿Puede incrementarse el nivel de multiprogramación? Con precaución, pero no es necesario. Podrías intentar agregar algún proceso más, pero corres el riesgo de saturar la memoria y provocar fallos de página. Con un 87% de uso de CPU, el sistema ya está muy bien aprovechado.
- ¿La paginación es útil? Sí. La paginación está permitiendo ejecutar los procesos sin sobrecargar el disco, manteniendo un alto rendimiento (throughput).

c) CPU: 13% |

Dispositivo de Paginación: 3%

- ¿Qué sucede?: El sistema está subutilizado. La CPU está ociosa (13%) y el disco de paginación también está "aburrido" (3%). Esto significa que los procesos actuales no están generando suficiente carga de trabajo para la CPU (quizás están esperando eventos de E/S de otros dispositivos lentos como teclado o red, o simplemente son pocos).
- ¿Puede incrementarse el nivel de multiprogramación? SÍ, definitivamente. Debes cargar más procesos en memoria. Como el disco de paginación está casi libre, tienes margen para agregar más carga sin riesgo inmediato de thrashing, y así lograrás que la CPU tenga más trabajo y suba ese 13%.
- ¿La paginación es útil? Actualmente no es un factor limitante, pero el mecanismo está disponible para soportar más carga cuando agregues procesos.

14. Considere un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda. Considere las siguientes medidas de utilización: Utilización del procesador: 20%

Utilización del dispositivo de paginación: 97,7%

Utilización de otros dispositivos de E/S: 5%

Cuales de las siguientes acciones pueden mejorar la utilización del procesador: a) Instalar un procesador más rápido

b) Instalar un dispositivo de paginación mayor

c) Incrementar el grado de multiprogramación

d) Instalar mas memoria principal

e) Decrementar el quantum para cada proceso

d) Instalar más memoria principal

- CORRECTO. Esta es la solución ideal. Al tener más RAM (más marcos físicos), los procesos pueden cargar sus "Working Sets" completos en memoria sin tener que expulsar páginas constantemente.
  - Resultado: Bajan los fallos de página El disco descansa Los datos están listos en RAM La CPU empieza a trabajar y sube su porcentaje de uso**.**

15. La siguiente fórmula describe el tiempo de acceso efectivo a la memoria al utilizar paginación para la implementación de la memoria virtual:

$$TAE = At + [(1 - p) * Am] + [p * (Tf + Am)]$$

Donde:

- **TAE = tiempo de acceso efectivo**
- **p = tasa de fallo de página (0 <= p <=1)**
- **Am = tiempo de acceso a la memoria real**
- **Tf = tiempo se atención de una fallo de pagina**
- **At = tiempo de acceso a la tabla de páginas. Es igual al tiempo de acceso a la memoria (Am) si la entrada de la tabla de páginas no se encuentra en la TLB.**

Suponga que tenemos una memoria virtual paginada, con tabla de páginas de 1 nivel, y donde la tabla de páginas se encuentra completamente en la memoria. Servir una falla de página tarda 300 nanosegundos si hay disponible un marco vacío o si la página reemplazada no se ha modificado, y 500 nanosegundos si se ha modificado. El tiempo de acceso a memoria es de 20 nanosegundos y el de acceso a la TLB es de 1 nanosegundo

a. Si suponemos una tasa de fallos de página de 0,3 y que siempre contamos con un marco libre para atender el fallo ¿Cuál será el TAE si el 50% de las veces la entrada de

la tabla de páginas se encuentra en la TLB (hit)?

b. Si suponemos una tasa de fallos de página de 0,3; que el 70% de las ocasiones la página a reemplazar se encuentra modificada. ¿Cuál será el TAE si el 60% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit)?

c. Si suponemos que el 60% de las veces la página a reemplazar está modificada, el 100% de las veces la entrada de la tabla de páginas requerida se encuentra en la TLB (hit) y se espera un TAE menor a 200 nanosegundos.

d. ¿Cuál es la máxima tasa aceptable de fallas de página?

16. Considere el siguiente programa:

```
#define Size 64
int A[Size; Size], B[Size; Size], C[Size;
Size]; int register i, j;
for (j = 0; j < Size; j ++)
     for (i = 0; i < Size; i++)
          C[i; j] = A[i; j] + B[i; j];
```

Si asumimos que el programa se ejecuta en un sistema que utiliza paginación por demanda para administrar la memoria, donde cada página es de 1Kb. Cada número entero (int) ocupa 4 bytes. Es claro que cada matriz requiere de 16 páginas para almacenarse. Por ejemplo: A[0,0]..A[0,63], A[1,0]..A[1,63], A[2,0]..A[2,63] y A[3,0]..A[3,63] se almacenará en la primera página.

Asumamos que el sistema utiliza un working set de 4 marcos para este proceso. Uno de los 4 marcos es utilizado por el programa y los otros 3 se utilizan para datos (las matrices). También consideramos que para los índices "i" y "j" se utilizan 2 registros, por lo que no es necesario el acceso a la memoria para estas 2 variables.

a. Analizar cuántos fallos de páginas ocurren al ejecutar el programa (considere las veces que se ejecuta C[i,j] = A[i,j] + B[i,j])

b. Puede ser modificado el programa para minimizar el número de fallos de páginas. En caso de ser posible indicar la cantidad de fallos de páginas que ocurren.

Versión	Orden de Recorrido	Fallos de Página	Explicación
a. Original	Por Columnas (j, i)	3072	Rompe la localidad espacial. Carga y descarga páginas

Versión	Orden de Recorrido	Fallos de Página	Explicación
			constantemente.
b. Optimizado	Por Filas (i, j)	48	Respeta el almacenamiento. Aprovecha cada página al máximo antes de cambiar.

17. Considere las siguientes secuencias de referencias a páginas de los procesos A y B, donde se muestra en instante de tiempo en el que ocurrió cada una (1 a 78):

Proceso A
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 1 2 1 3 4 3 3 2 1 1 3 4 5 6
1 2 1 3 3 4 5 6 6 6 5 4 3 1 1 2 3 4 5 4 3 2 1 1 1 ^{40} ^{41} ^{42} ^{43} ^{44} ^{45} ^{46} ^{47} ^{48} ^{49} ^{50} 5 1 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69
70 71 72 73 74 75 76 77 78 2 2 1 2 1 1 2 3 4 5 6 7 8 6 5 3 1 2 3 4 5 6 5 1 1 2 3 4 5 4 2 1 1 2 3 4 5 1 1

Proceso B
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 1 2 3 3 3 4 4 5 1 1 1 2 3 4
4 4 4 1 1 2 3 6 5 6 5 4 6 1 1 1 1 4 5 1 3 2 1 1 2 ^{40} ^{41} ^{42} ^{43} ^{44} ^{45} ^{46} ^{47} ^{48} ^{49} ^{50} 5 1 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69
70 71 72 73 74 75 76 77 78 2 2 3 3 4 4 4 3 4 5 4 6 1 1 1 2 3 3 3 4 5 5 5 1 6 2 3 1 2 3 1 1 1 2 2 2 2 3 3

a. Considerando una ventana Δ=5, indique cuál sería el conjunto de trabajo de los procesos A y B en el instante 24 ($WS_A(24)$ y $WS_B(24)$)
b. Considerando una ventana Δ=5, indique cuál sería el conjunto de trabajo de los procesos A y B en el instante 60 ($WS_A(60)$ y $WS_B(60)$)
c. Para el los WS obtenidos en el inciso a), si contamos con 8 frames en el sistema ¿Se puede indicar que estamos ante una situación de trashing? ¿Y si contáramos con 6 frames?
d. Considerando únicamente el proceso A, y suponiendo que al mismo se le asignaron

inicialmente 4 marcos, donde el de reemplazo de páginas es realizado considerando el algoritmo FIFO. ¿Cuál será la tasa de fallos en el instante 38 de páginas suponiendo que la misma se calcula contando los fallos de páginas que ocurrieron en las últimas 10 unidades de tiempo?

e. Para el valor obtenido en el inciso d), si suponemos que el S.O. utiliza como límites superior e inferior de tasa de fallos de páginas los valores 2 y 5 respectivamente ¿Qué acción podría tomar el Kernel respecto a la cantidad de marcos asignados al proceso?

Administración de E/S

18. Dispositivos

a. Los dispositivos, según la forma de transferir los datos, se pueden clasificar en 2 tipos:

i. Orientados a bloques

ii. Orientados a flujos

b. Describa las diferencias entre ambos tipos.

Característica	Dispositivos de Bloque	Dispositivos de Carácter (Flujo)
Unidad de Info	Almacenan la información en bloques de tamaño fijo (ej: 512 bytes a 4KB).	Transfieren la información como un flujo continuo de bytes, sin estructura de bloque definida.
Direccionamiento	Son direccionables. Puedes leer o escribir un bloque específico independientemente de los demás (acceso aleatorio/random access).	No son direccionables. No puedes pedir "el byte número 1000" sin haber leído los 999 anteriores. Solo tienen acceso secuencial.
Transferencia	Se leen/escriben bloques completos de una sola vez.	Se transfiere byte a byte (o carácter a carácter).
Uso Principal	Generalmente usados para almacenamiento persistente de datos (FileSystems).	Usados para interacción con humanos, comunicación o dispositivos simples.

c. Cite ejemplos de dispositivos de ambos tipos.

Orientados a Bloques:

- Discos Duros (HDD).
- Unidades de Estado Sólido (SSD).
- Memorias USB (Pendrives).

- Discos Ópticos (CD-ROM, DVD).

Orientados a Flujos (Caracteres):

- Teclado.
- Mouse.
- Impresoras.
- Tarjetas de Red (aunque a veces tienen un tratamiento híbrido, se comportan como flujos de datos).
- Consola / Terminal.

d. Enuncie las diferencias que existen entre los dispositivos de E/S y que el Kernel debe considerar.

El Kernel debe ser capaz de gestionar una gran variedad de dispositivos, por lo que debe considerar las siguientes diferencias críticas para uniformar su acceso (normalmente a través de *Drivers*):

1. **Velocidad de Transferencia (Data Rate):** El Kernel no puede tratar igual a un teclado (que envía 10 bytes por segundo) que a un disco NVMe (que envía 5 GB por segundo). Necesita *buffers* y mecanismos de interrupción distintos.
2. **Unidad de Transferencia:** ¿El dispositivo envía un byte a la vez o un bloque de 4KB? El Kernel debe saber cómo empaquetar o desempaquetar esos datos.
3. **Representación de Datos:** Diferentes dispositivos pueden usar diferentes codificaciones de caracteres o protocolos de paridad.
4. **Condiciones de Error:** La forma en que un disco reporta un error de lectura es muy distinta a cómo una impresora reporta falta de papel. El Kernel debe unificar estos reportes para las aplicaciones.
5. **Acceso Compartido vs. Dedicado:** Un disco puede ser accedido por varios procesos a la vez (concurrencia gestionada por el FS), pero una impresora o un teclado generalmente deben ser asignados de forma exclusiva a un usuario o proceso momentáneamente.

19. Técnicas de E/S. Describa cómo trabajan las siguientes técnicas de E/S.

a. E/S programada
b. E/S dirigida por interrupciones
c. DMA (Acceso Directo a Memoria)

a. E/S Programada (Programmed I/O)

Es la técnica más antigua y sencilla, pero también la más ineficiente.

- Cómo funciona: El procesador (CPU) tiene el control total. Envía una orden al módulo de E/S y luego se queda en un bucle "preguntando" constantemente al dispositivo: *"¿Ya terminaste? ¿Ya terminaste?"*. Esto se conoce como espera activa (busy waiting) o polling.
- Problema: La CPU pierde muchísimo tiempo valioso esperando a dispositivos lentos (como una impresora o un disco) en lugar de ejecutar otros procesos.
- Analogía: Es como pedir una pizza y quedarte parado en la puerta mirando al repartidor hasta que llegue, sin hacer nada más en tu casa mientras tanto.

b. E/S Dirigida por Interrupciones (Interrupt-Driven I/O)

Esta técnica soluciona el problema de la "espera activa".

- Cómo funciona: La CPU envía la orden al módulo de E/S y se va a hacer otra cosa (ejecutar otro proceso). Cuando el dispositivo termina su tarea (o tiene un dato listo), le envía una señal eléctrica (interrupción) a la CPU para avisarle. La CPU detiene lo que estaba haciendo, atiende la transferencia del dato, y luego vuelve a su trabajo.
- Ventaja: La CPU es mucho más eficiente porque no espera.
- Desventaja: Aunque la CPU no espera, todavía es responsable de mover los datos byte por byte entre el dispositivo y la memoria. Si hay que transferir un archivo grande, la CPU es interrumpida miles de veces, lo cual degrada el rendimiento (overhead).
- Analogía: Pides la pizza y te pones a estudiar. Cuando suena el timbre (interrupción), dejas de estudiar, recibes la pizza y luego vuelves a estudiar.

c. DMA (Acceso Directo a Memoria / Direct Memory Access)

Es la técnica más avanzada y eficiente, diseñada para mover grandes volúmenes de datos.

- Cómo funciona: Se añade un chip especial llamado Controlador DMA. La CPU le dice al DMA: *"Mueve este bloque de 10 MB del disco a la memoria RAM y avísame cuando termines todo"*.
    - La CPU se desentiende y sigue trabajando.
    - El chip DMA toma el control del bus y transfiere todo el bloque directamente a la memoria sin molestar a la CPU.
    - Solo cuando todo el bloque se ha copiado, el DMA interrumpe a la CPU una sola vez.
- Ventaja: Elimina la carga de trabajo de la CPU casi por completo en transferencias de disco, red o multimedia.
- Analogía: Tienes un asistente (DMA). Le dices "recibe las pizzas y ponlas en la mesa". Tú sigues estudiando sin interrupciones hasta que el asistente te avisa que la cena está servida.

20. La técnica de E/S programa puede trabajar de dos formas:

a. E/S mapeada

En esta técnica, los registros de los dispositivos y la memoria RAM comparten el mismo espacio de direcciones.

- Cómo trabaja: El procesador no distingue entre "hablar con la memoria" y "hablar con un disco". Si escribes un dato en la dirección 0xA000, podrías estar enviando un comando a la impresora en lugar de guardar un número en la RAM.
- Instrucciones: Se utilizan las mismas instrucciones estándar de lectura/escritura de memoria (como MOV, LOAD, STORE). No hacen falta instrucciones especiales.
- Ventaja: Es más flexible y permite usar toda la potencia del set de instrucciones de la CPU sobre los dispositivos.
- Desventaja: "Roba" parte del espacio de direcciones que podría usarse para memoria RAM.

b. E/S aislada

En esta técnica, los dispositivos tienen su propio espacio de direcciones, separado totalmente de la memoria RAM.

- Cómo trabaja: La dirección 10 de memoria RAM es diferente al Puerto 10 de E/S. El procesador tiene que usar líneas de control específicas en el bus para indicar si quiere acceder a memoria o a E/S.

- Instrucciones: Se requieren instrucciones especiales exclusivas para E/S (típicamente IN y OUT en ensamblador x86).
- Ventaja: No ocupa espacio de memoria RAM; el mapa de memoria queda intacto para los programas.
- Desventaja: Es más rígido y requiere instrucciones específicas en el procesador.

Indique cómo trabajan estas 2 técnicas.

21. Enuncie las metas que debe perseguir un SO para la administración de la entrada/salida.

Las principales metas que debe perseguir el Kernel son:

1. Independencia del Dispositivo (Abstracción):
    - Es la meta más importante. Los programas de usuario no deben saber qué tipo de dispositivo están usando.
    - Deben poder usar las mismas instrucciones (open, read, write) para leer un archivo de un disco duro, de un pendrive USB o de un DVD. El S.O. se encarga de traducir esas órdenes genéricas a las órdenes específicas de cada hardware.
2. Eficiencia:
    - Los dispositivos de E/S son mucho más lentos que la CPU.
    - El objetivo del S.O. es evitar que la CPU pierda tiempo esperando a los dispositivos. Para esto utiliza técnicas como el Buffering (almacenamiento intermedio), el Caching (memoria rápida) y el Spooling (colas de impresión).
3. Nombramiento Uniforme (Uniform Naming):
    - El nombre de un archivo o dispositivo debe ser independiente de la máquina.
    - Por ejemplo, en UNIX/Linux, los dispositivos aparecen como archivos en el directorio /dev (ej: /dev/sda1), permitiendo acceder a ellos de forma estándar.
4. Manejo de Errores:
    - Los errores deben manejarse lo más cerca posible del hardware.
    - Si un disco falla al leer un sector, el S.O. (a través del driver) debe intentar releerlo varias veces antes de molestar al usuario con un error. Solo si falla repetidamente debe informar al nivel superior.
5. Manejo de Dispositivos Compartidos y Dedicados:
    - El S.O. debe resolver problemas de concurrencia.
    - Compartidos: Como un disco rígido, donde varios procesos pueden leer/escribir

"al mismo tiempo" (el S.O. planifica los turnos).

- Dedicados: Como una impresora, donde el S.O. debe garantizar que solo un proceso la use a la vez para que no se mezclen las hojas.

22. Drivers

a. ¿Qué son?

Un driver (o controlador) es un módulo de software específico que actúa como un intermediario o traductor entre el Sistema Operativo (Kernel) y un dispositivo de hardware físico.

- Su función principal es ocultar las complejidades y detalles técnicos del hardware, ofreciendo al S.O. una interfaz estándar para usarlo.
- Sin el driver, el S.O. no sabría cómo "hablar" con la placa de video, el disco o la impresora, ya que cada marca y modelo tiene sus propios registros y comandos internos.

b. ¿Qué funciones mínimas deben proveer?

Para que un driver sea útil y el Kernel pueda usarlo, debe implementar al menos las siguientes funciones básicas (la "Interfaz del Driver"):

1. Inicialización (Open/Init): Preparar el dispositivo para funcionar cuando arranca o cuando se conecta (encenderlo, verificar que responda).
2. Lectura (Read): Aceptar una solicitud del Kernel para traer datos desde el dispositivo hacia la memoria.
3. Escritura (Write): Aceptar datos de la memoria para enviarlos al dispositivo.
4. Control (Ioctl): Configurar parámetros específicos (ej: cambiar la resolución de pantalla, expulsar el CD, rebobinar cinta).
5. Manejo de Interrupciones (ISR): Saber qué hacer cuando el dispositivo avisa que terminó una tarea o que ocurrió un error.

c. ¿Quién determina cuáles deben ser estas funciones?

Las funciones las determina el diseñador del Sistema Operativo (el Kernel), no el fabricante del hardware.

- El S.O. dice: *"Si quieres conectar un disco a mi sistema (Windows, Linux, etc.), tu driver TIENE que tener una función llamada read_block y otra write_block que reciban estos parámetros exactos"*.
- El fabricante del hardware (ej: NVIDIA, HP, Intel) es quien escribe el código del driver, pero debe obedecer estrictamente la "plantilla" o interfaz que definió el Sistema Operativo.

23. Realice un gráfico que marque la relación entre el Subsistema de E/S, los drivers, los controladores de dispositivos y los dispositivos.

El flujo de una operación de E/S viaja a través de estas capas:

1. **Subsistema de E/S (Kernel):** Es la capa superior dentro del S.O. Provee la interfaz genérica e independiente del dispositivo a las aplicaciones. Se encarga de la gestión de buffers, caching, spooling y planificación general.

2. **Drivers (Manejadores de Dispositivos):** Es el software específico de cada hardware. Traduce las órdenes genéricas del Subsistema de E/S (ej: "leer bloque n") a comandos de bajo nivel entendibles por el controlador específico (ej: "escribir en registro 0x3F").

3. **Controladores de Dispositivos (Hardware):** Es el componente electrónico (placa o chip) que gobierna al dispositivo. Recibe los comandos binarios del driver y genera las señales eléctricas para mover las partes físicas.

4. **Dispositivos (Físico):** Es la unidad mecánica o electrónica real (el motor del disco, el láser de la impresora) que realiza la acción.

24. un ejemplo los pasos mínimos que se suceden desde que un proceso genera un Describa mediante requerimiento de E/S hasta que el mismo llega al dispositivo.

Escenario de Ejemplo: Un programa de procesamiento de texto (como el Bloc de Notas o Word) quiere guardar un archivo en el disco rígido.
Paso 1: El Proceso (Espacio de Usuario)
El usuario hace clic en "Guardar". El programa ejecuta una llamada al sistema (System Call), por ejemplo write(), solicitando al Sistema Operativo que escriba datos en un archivo específico.
Acción: El programa se detiene momentáneamente y cede el control al Kernel.
Paso 2: Subsistema de E/S (Kernel - Independiente del Dispositivo)
El Kernel recibe la solicitud.
Verifica permisos (¿tiene permiso de escritura?).
Verifica si el archivo ya está abierto.
Traduce el nombre del archivo (ej: "t.txt") a una dirección lógica en el sistema de archivos (ej: bloques lógicos).
Decide a qué Driver corresponde esa solicitud (ej: Driver del Disco SATA).
Paso 3: El Driver (Manejador de Dispositivo)
El Kernel llama a la función de escritura del Driver específico del disco.
El Driver traduce la petición lógica ("escribir en el bloque lógico 100") a comandos físicos que el hardware entienda (ej: "Cilindro 5, Cabeza 1, Sector 3").
El Driver verifica si el dispositivo está libre. Si está ocupado, pone la petición en una cola.
Cuando es el turno, el Driver escribe en los registros del Controlador (E/S Programada o Mapeada) para darle la orden.
Paso 4: El Controlador del Dispositivo (Hardware Electrónico)
El chip controlador en la placa del disco recibe los comandos en sus registros.
Interpreta la orden: "Mover el brazo a la pista 5 y escribir estos datos".
Convierte estas órdenes digitales en señales eléctricas (voltaje).
Paso 5: El Dispositivo (Hardware Físico)
El mecanismo físico actúa.
El brazo mecánico del disco se mueve (Seek).
El cabezal magnético escribe los bits en la superficie del plato giratorio.

Nota: Una vez que el dispositivo termina (Paso 6, aunque el ejercicio pide "hasta que llega"), el Controlador genera una Interrupción para avisarle al Driver "¡Ya terminé!", y el camino vuelve hacia arriba hasta despertar al proceso original.

26. Enuncie qué servicios provee el Kernel para la administración de E/S.

- Planificación de E/S (I/O Scheduling):
  - El Kernel no siempre atiende las peticiones en el orden que llegan (FIFO). A menudo las reordena para mejorar la eficiencia global.
  - *Ejemplo:* En un disco rígido, el Kernel reordena las solicitudes de lectura para minimizar el movimiento del brazo mecánico (como veremos en el ejercicio 31).
- Buffering (Almacenamiento Intermedio):
  - Consiste en usar un área de memoria (buffer) para almacenar datos mientras se transfieren entre dos dispositivos o entre un dispositivo y una aplicación.
  - *Función:* Sirve para suavizar las diferencias de velocidad. Si internet baja datos lento pero el disco graba rápido, el buffer acumula datos hasta tener suficiente para escribir de una vez.
- Caching (Caché):
  - Mantiene en memoria rápida (RAM) una copia de los datos que están en un dispositivo lento (Disco).
  - *Función:* Si un proceso pide un dato que ya está en la caché, se lo entrega al instante sin tener que acceder al disco físico, mejorando drásticamente el rendimiento.
- Spooling (Simultaneous Peripheral Operations On-line):
  - Es un buffer que mantiene la salida para un dispositivo que no puede aceptar flujos de datos intercalados (como una impresora).
  - *Función:* Si tres usuarios imprimen a la vez, el Kernel guarda los trabajos en el disco (Spool) y los manda a la impresora uno por uno, evitando que salgan hojas con líneas mezcladas de diferentes documentos.
- Reserva de Dispositivos (Device Reservation):
  - Provee mecanismos para que un proceso obtenga acceso exclusivo a un dispositivo (candados/locks). Esto es vital para dispositivos dedicados (como unidades de cinta o impresoras) para evitar conflictos.

 Manejo de Errores (Error Handling):

- El Kernel intercepta los fallos de hardware.
- Si es un error transitorio (ej: fallo de lectura en disco), el Kernel puede reintentar la operación automáticamente. Si el error persiste, informa al proceso de usuario de manera controlada.

Administración de Discos

27. Describa en forma sintética, cómo es la organización física de un disco, puede utilizar gráficos para mayor claridad.

Un disco rígido es un dispositivo electromecánico compuesto por una pila de discos magnéticos giratorios. Su organización se jerarquiza de la siguiente manera:

1. Platos (Platters): Son los discos circulares físicos (generalmente de aluminio o cristal) recubiertos de material magnético. Giran a gran velocidad (ej: 7200 RPM). Un disco duro puede tener uno o varios platos apilados en un eje común.
2. Caras (Surfaces/Heads): Cada plato tiene dos caras (superior e inferior). Para leer cada cara, existe un Cabezal (Head) de lectura/escritura situado en el extremo de un brazo mecánico.
    - *Nota:* Si hay 4 platos, usualmente hay 8 caras y 8 cabezales.
3. Pistas (Tracks): Cada cara del plato está dividida en círculos concéntricos (como los anillos de un árbol o un disco de vinilo) llamados pistas.
4. Cilindros (Cylinders): Es un concepto vertical abstracto. Un cilindro está formado por todas las pistas que están a la misma distancia del centro en todos los platos apilados.
    - *Importancia:* El brazo mecánico mueve todos los cabezales a la vez. Cuando el brazo se detiene en una posición, puede leer todas las pistas de ese "cilindro" sin volver a moverse, lo cual es muy rápido.
5. Sectores (Sectors): Cada pista se divide en pequeños arcos llamados sectores. Es la unidad mínima de almacenamiento direccionable.
    - *Tamaño estándar:* Históricamente 512 bytes, aunque los discos modernos usan 4096 bytes (4KB).

28. La velocidad promedio para la obtención de datos de un disco está dada por la suma de los siguientes tiempos:

- ➢ ***Seek Time***
- ➢ ***Latency Time***
- ➢ ***Transfer Time***

De una definición para cada uno.

La velocidad promedio para obtener un dato se compone de la suma de estos tres tiempos:

1. Seek Time (Tiempo de Búsqueda):
    - Definición: Es el tiempo que tarda el brazo mecánico del disco en mover los cabezales desde su posición actual hasta el cilindro (o pista) donde se encuentra el dato deseado.
    - *Nota:* Es el componente más lento de los tres, ya que implica mover piezas físicas mecánicas. Minimizar este tiempo es la clave de los algoritmos de planificación (como SCAN o SSTF).
2. Latency Time (Tiempo de Latencia o Latencia Rotacional):
    - Definición: Una vez que el cabezal llegó a la pista correcta, es el tiempo que hay que esperar hasta que el disco gire y el sector deseado pase justo por debajo del cabezal.
    - *Nota:* En promedio, es el tiempo que tarda el disco en dar media vuelta. Depende exclusivamente de las RPM (Revoluciones Por Minuto) del disco.
3. Transfer Time (Tiempo de Transferencia):
    - Definición: Es el tiempo que tarda el disco en leer los bits magnéticos del sector y enviarlos a través del cable (SATA/SAS) hacia la memoria, o viceversa.
    - *Nota:* Es el más rápido de los tres. Depende de la densidad de bits en el disco y de la velocidad de rotación.

29. Suponga un disco con las siguientes características:

➢ 7 platos con 2 caras utilizables cada uno.
➢ 1100 cilindros, con 300 sectores por pista, donde cada sector es 512 bytes. ➢ Seek Time de 10 ms
➢ 9000 RPM .
➢ Velocidad de Transferencia de 10 MiB/s (Mebibytes por segundos).

a. Calcule la capacidad total del disco.

Capacidad en Bytes: 2.365.440.000 bytes
Capacidad en GigiBytes= 2.2 Gib

b. ¿Cuántos sectores de disco se requerirían si se quiere almacenar 3 MiB(MebiBytes) de datos?

El sector del disco tiene 512 bytes, entonces para almacenar 3MiB (3.145.728 bytes) se necesitan 3.145.728/512= 6144 sectores del disco.

c. Calcule el tiempo de latencia.

9000/60 s= 150 revoluciones por segundo
Como el tiempo de latencia se promedia el tiempo que tarde la cabecera en dar una vuelta, entonces el tiempo de latencia es 1/(2*150)=0.00333 segundos

30. Originalmente, los sectores de un disco se identificaban por la terna de valores (C,H,S) esto es Cylinder-head-sector (Cilindro, Cabeza , Sector), con C desde 0 hasta la cantidad de cilindros menos 1, H desde 0 hasta la cantidad de cabezas lecto/grabadoras menos 1 y S desde 1 hasta la cantidad de sectores por pista.

En los '90 los controladores de disco reemplazaron la identificación de los sectores por un modelo numérico denominado LBA (Logical Block Addressing). La numeración

comenzaba en 0 y alcanzaba la cantidad total de sectores menos 1. Cada sector en formato (C,H,S) se puede traducir al LBA a partir de la siguiente fórmula:

$$A = (C * N_{heads} + H) * N_{sectors} + (S -1)$$

Donde:

- ➔ **A es el valor LBA a obtener**
- ➔ **N_{heads} es la cantidad de cabezas lecto/grabadoras**
- ➔ **$N_{sectors}$ es la cantidad de sectores por pista**

También se puede obtener el valor de la terna (C,H,S) a partir de un valor LBA con las

siguientes fórmulas:

C = LBA div (N_{heads} * $N_{sectors}$)	***H =Resto de (LBA mod (N_{heads} * $N_{sectors}$)) / $N_{sectors}$***	***S = (LBA mod (N_{heads} * $N_{sectors}$)) mod $N_{sectors}$ + 1***

30) **Si se tiene un disco con 1020 cilindros, 16 cabezas y 63 sectores por pista, calcule la cantidad total de sectores que tendrá el disco y determine el valor LBA para las siguientes ternas (C,H,S):**
Hay un total de 1.028.160 sectores.

i. (3, 2, 1)
A= (C*Nheads + H) * Nsectores +(S-1)= 3150

ii. (0, 0, 1)
A= 0

iii. (1019, 15, 63)
A= 1.028.159

iv. (500, 8, 44)
A= 504.547

b. Determine el valor de la terna (C,H,S) para los siguientes valores LBA:

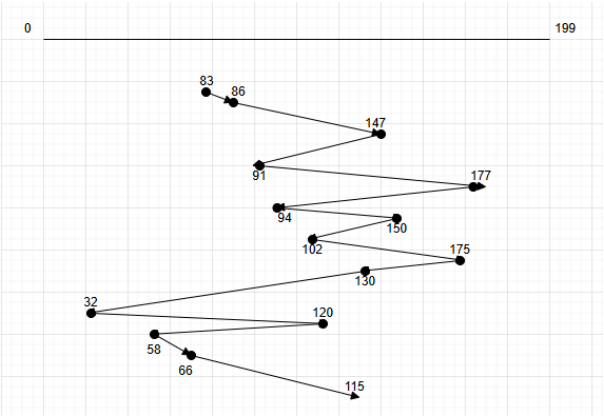
i. 1 (0,0,2)
ii. 100 (0,1,38)
iii. 14890 (14,12,23)
iv. 45687 (45,5,13)

31. **El *Seek Time* es el parámetro que posee mayor influencia en el tiempo real necesario para transferir datos desde o hacia un disco. Es importante que el Kernel planifique los diferentes requerimientos al disco para minimizar el movimiento de la cabeza lecto-grabadora.**

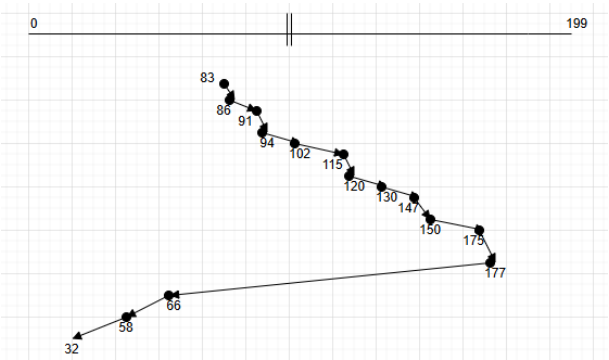
Analicemos las diferentes políticas de planificación de requerimientos a disco con un ejemplo: Supongamos un *Head* con movimiento en 200 *tracks* (numerados de 0 a 199), que está en el track 83 atendiendo un requerimiento y anteriormente atendió un requerimiento en el track 75.

Si la cola de requerimientos es: {86, 147, 91, 177, 94, 150, 102, 175, 130, 32, 120, 58, 66, 115}. Realice los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

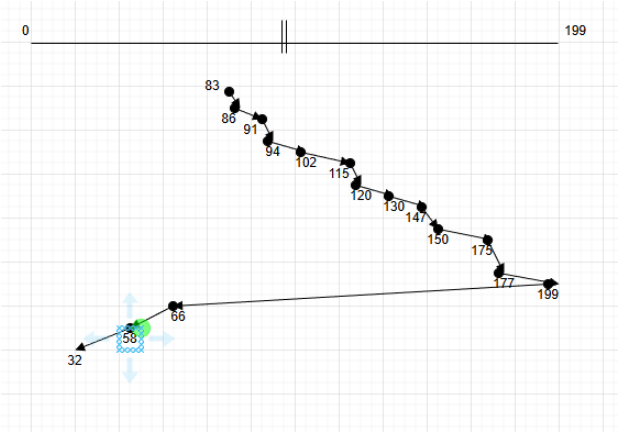
a. FCFS (*First Come, First Served*)



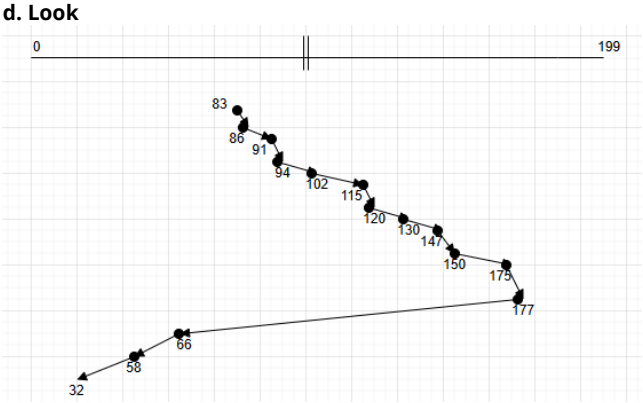
b. SSTF (*Shortest Seek Time First*)



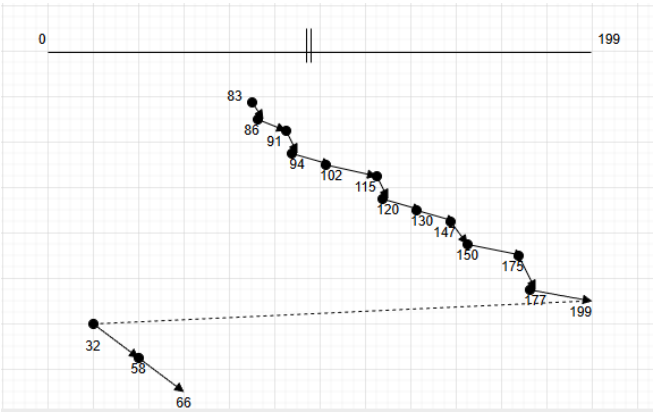
c. Scan



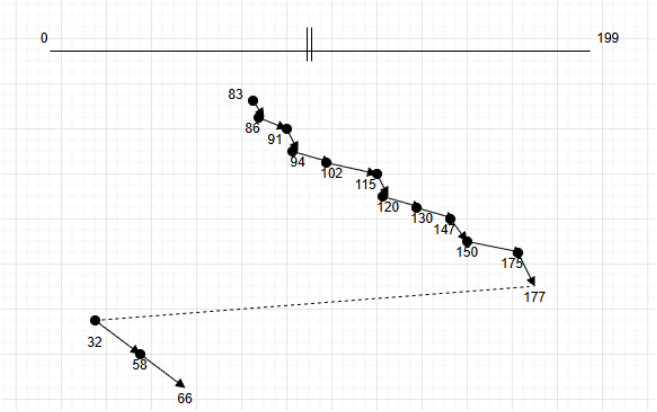
d. Look



e. C-Scan (*Circular Scan*)



f. C-Look (*Circular Look)*



32. ¿Alguno de los algoritmos analizados en el ejercicio anterior pueden causar inanición de requerimientos?

SSTF (Shortest Seek Time First)

Este algoritmo siempre elige el pedido que está más cerca de la posición actual.

- ¿Cómo ocurre la Inanición? Imagina que el brazo está en la pista 50 y hay un pedido en la pista 200 (muy lejos). Si constantemente entran nuevos pedidos "cercanos" (ej: 51, 49, 52, 48, 53...), el brazo se quedará oscilando en esa zona atendiendo a los nuevos y nunca viajará hasta el 200.

- El pedido lejano muere de "hambre" (starvation) porque siempre hay alguien más cerca que se "cuela" en la fila.

2. Los Justos: SCAN, C-SCAN, LOOK y C-LOOK

Estos algoritmos NO sufren de inanición (o es muy improbable).

- ¿Por qué? Porque funcionan como un barrido sistemático. El brazo se mueve obligatoriamente en una dirección hasta el final (o hasta el último pedido).

- Aunque lleguen miles de pedidos nuevos en el medio, el brazo eventualmente pasará por todas las pistas. Tarde o temprano, le tocará el turno al pedido de la pista 200.

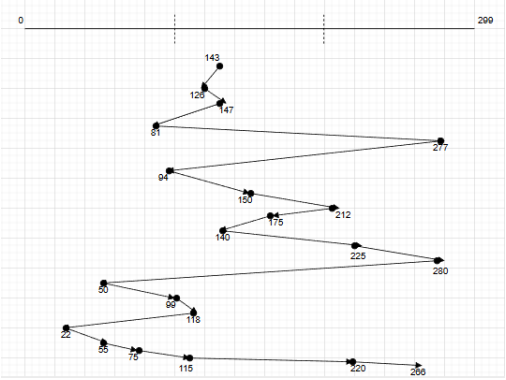
3. El Imparcial: FCFS

- NO causa inanición porque atiende en estricto orden de llegada. Aunque sea lento, te garantiza que si llegaste, te van a atender.

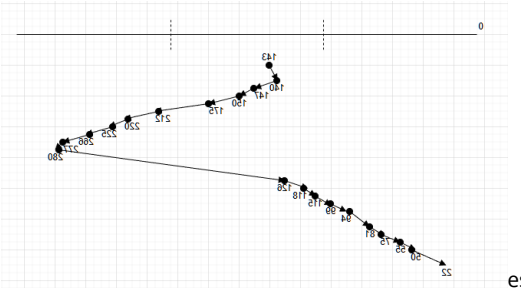
33. Supongamos un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 143 atendiendo un requerimiento y anteriormente atendió un requerimiento en la pista 125.

Si la cola inicial de requerimientos es: {126, 147, 81, 277, 94, 150, 212, 175, 140, 225, 280, 50, 99, 118, 22, 55} y luego de 30 movimientos se incorporan los requerimientos de las pistas {75, 115, 220 y 266}. Realice los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos.

a. FCFS

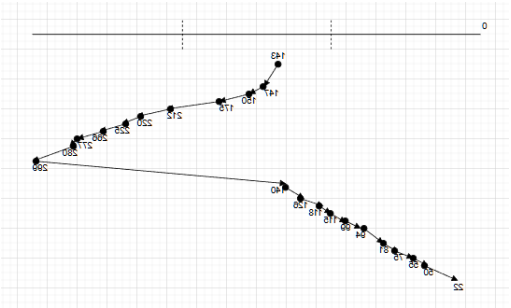


b. SSTF

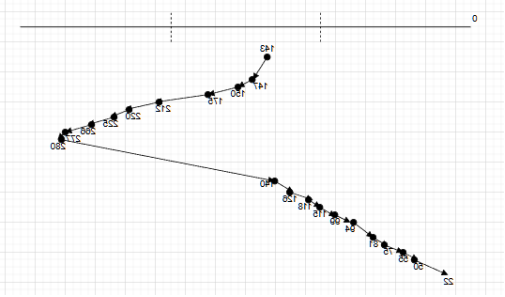


esta MAL

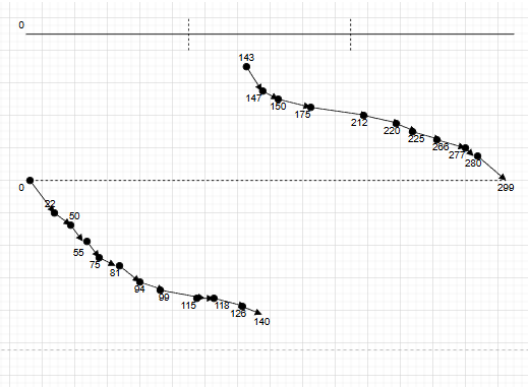
c. Scan



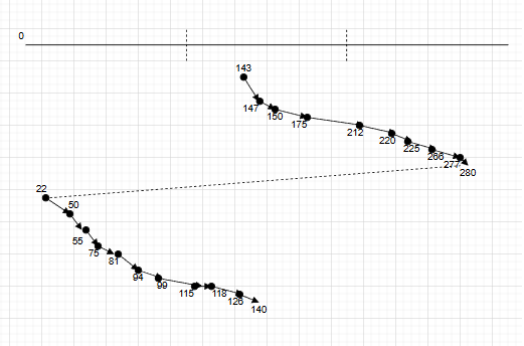
d. Look



e. C-Scan



f. C-Look

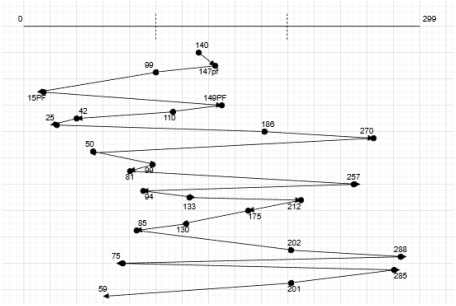


34. Supongamos un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 140 atendiendo un requerimiento y anteriormente atendió un

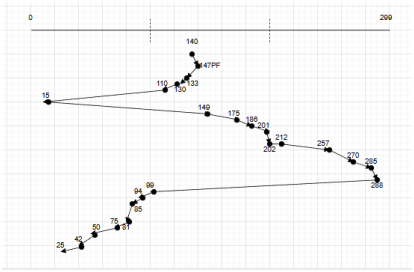
requerimiento en la pista 135.

Si la cola de requerimientos es: {99, 110, 42, 25, 186, 270, 50, 99, 147^{PF}, 81, 257, 94, 133, 212, 175, 130} y después de 30 movimientos se incorporan los requerimientos de las pistas {85, 15^{PF}, 202 y 288} y después de otros 40 movimientos más se incorporan los requerimientos de las pistas {75, 149^{PF}, 285, 201 y 59}. Realice los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

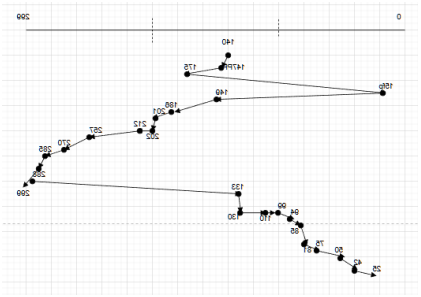
a. FCFS



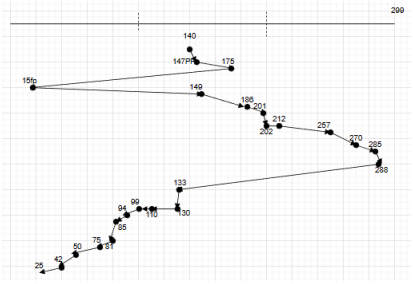
b. SSTF



c. Scan

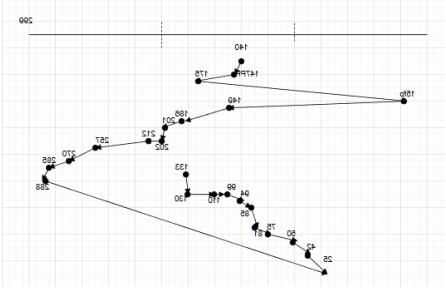


d. Look



e. C-Scan

f. C-Look



Administración de Archivos

35. Dados las siguientes técnicas de administración de espació de un archivo:

➢ **Asignación contigua**

a. Descripción (Cómo trabaja): El archivo ocupa un conjunto de bloques consecutivos en el disco.
Analogía: Es como grabar en un cassette o reservar butacas en el cine: si el archivo necesita 5 bloques, el S.O. busca 5 espacios libres uno al lado del otro.
El directorio solo necesita guardar: Dirección de inicio y Longitud.
b. Ventajas y Desventajas:
Ventajas:
Es excelente para la lectura secuencial (el cabezal del disco casi no se mueve).
Es muy sencillo de implementar.
Desventajas:
Difícil encontrar espacio: Si querés guardar un archivo grande, necesitás un "hueco" grande continuo.

Problema de crecimiento: Si el archivo quiere crecer y no tiene espacio libre al lado, tenés que mover *todo* el archivo a otro hueco más grande.
c. Fragmentación:
Produce Fragmentación Externa: Sí. Quedan huecos libres entre archivos que son muy chicos para ser usados por archivos nuevos.
Produce Fragmentación Interna: Sí (mínima), solo en el último bloque si el archivo no lo llena completo.

➢ **Asignación enlazada**

a. Descripción (Cómo trabaja): El archivo es una lista enlazada de bloques. Los bloques pueden estar dispersos por cualquier parte del disco.

- Analogía: Es una "búsqueda del tesoro". El directorio apunta al primer bloque. El primer bloque contiene datos y un puntero (flecha) que dice dónde está el segundo bloque, y así sucesivamente.
- El directorio guarda: Puntero al inicio y Puntero al final.

b. Ventajas y Desventajas:

- Ventajas:
  - No hay fragmentación externa: Cualquier bloque libre sirve.
  - El archivo puede crecer dinámicamente sin problemas mientras haya espacio en el disco.
- Desventajas:
  - Acceso aleatorio muy lento: Para leer el bloque 10, tenés que leer obligatoriamente el 1, 2, 3... hasta llegar al 10 para encontrar los punteros.
  - Desperdicio de espacio: En cada bloque hay que gastar unos bytes para guardar el puntero al siguiente (overhead).
  - Fiabilidad: Si se rompe un puntero o un bloque se daña, perdés el resto del archivo.

c. Fragmentación:

- Fragmentación Externa: NO.
- Fragmentación Interna: Sí, solo en el último bloque.

➢ **Asignación indexada**

a. Descripción (Cómo trabaja): Se resuelve el problema del acceso lento de la enlazada trayendo todos los punteros a un solo lugar: el Bloque Índice.

- Analogía: Es como el índice de un libro. Cada archivo tiene su propio "bloque índice" (una tabla). Ese bloque no guarda datos del usuario, sino una lista de direcciones: "El bloque 1 del archivo está en la pista 50; el bloque 2 está en la pista 88...".
- El directorio guarda: La dirección del Bloque Índice.

b. Ventajas y Desventajas:

- Ventajas:
  - Admite acceso directo (aleatorio) rápido sin sufrir fragmentación externa.
- Desventajas:
  - Desperdicio (Overhead): Si el archivo es muy chiquito (ej: 1 bloque), igual tenés que gastar un bloque entero extra solo para el índice.
  - Límite de tamaño: Si el archivo es gigante, un solo bloque índice no alcanza para guardar todas las direcciones (se requiere esquema de niveles como

en Unix, ver ejercicio 42).

c. Fragmentación:

- Fragmentación Externa: NO.
- Fragmentación Interna: Sí, en el último bloque de datos y también espacio desperdiciado en el bloque índice si no se usa completo.

36. Gestión de espacio libre. Dados los siguientes métodos de gestión de espacio libre en un disco:

➢ Tabla de bits

a. ¿Cómo trabaja? El disco se representa como un vector (array) gigante de bits, donde cada bit corresponde a un bloque del disco.

- Si el bit vale 1: El bloque está LIBRE.
- Si el bit vale 0: El bloque está OCUPADO.
- *Ejemplo:* 00111001 significaría que los bloques 2, 3, 4 y 7 están libres.

b. Ventajas y Desventajas

- Ventajas:
  - Búsqueda de Contigüidad: Es excelente para encontrar espacio contiguo. Si necesitas 5 bloques juntos, el procesador solo tiene que buscar una secuencia de cinco "1s" seguidos (11111) en el mapa, lo cual es muy rápido a nivel de hardware.
  - Simplicidad: Es muy fácil de entender e implementar.
- Desventajas:
  - Consumo de RAM: Para que sea eficiente, el mapa de bits debe estar cargado en la memoria principal (RAM). Si el disco es enorme y los bloques son chicos, el mapa puede ocupar varios megabytes de RAM valiosa.
  - Búsqueda lenta si está lleno: Si el disco está casi lleno, el S.O. debe escanear millones de "0s" hasta encontrar un "1", lo que puede demorar un poco (aunque los procesadores modernos tienen instrucciones especiales para esto).

➢ **Lista Enlazada**

a. ¿Cómo trabaja? Se mantiene una lista encadenada de todos los bloques libres.

- El Sistema Operativo tiene un puntero que dice: *"El primer bloque libre es el #50"*.
- Vas al bloque físico #50 y dentro de él (en sus primeros bytes) hay un dato que dice: *"El siguiente libre es el #102"*.
- Vas al #102 y dice: *"El siguiente es el #800"*, y así sucesivamente.

b. Ventajas y Desventajas

- Ventajas:
  - No gasta espacio extra: Utiliza los propios bloques libres del disco para guardar los punteros. No ocupa espacio "útil" ni consume RAM (solo guardas el puntero al primero).
  - Rápido para 1 bloque: Si solo necesitas *un* bloque cualquiera, tomas el primero de la lista y actualizas el puntero. Es instantáneo.
- Desventajas:
  - Ineficiente para Contigüidad: No tienes forma de saber si hay bloques juntos. La lista salta del bloque 50 al 8000 y vuelve al 2. No sirve para la *Asignación Contigua*.
  - Lento de recorrer: Si necesitas verificar cuántos bloques libres hay o buscar un bloque específico, tienes que leer el disco físicamente bloque por bloque (I/O), lo cual es lentísimo.
  - Fragilidad: Si se daña un bloque de la cadena, pierdes el rastro de todo el resto del espacio libre.

➢ **Agrupamiento**

a. Descripción (Cómo trabaja):

Es una mejora de la Lista Enlazada. En lugar de que un bloque libre solo apunte al "siguiente" bloque libre (1 a 1), aprovechamos que el bloque está vacío para guardar muchas direcciones.

- Si un bloque tiene 512 bytes, podemos guardar allí cientos de direcciones de otros bloques libres.
- El primer bloque libre guarda las direcciones de, por ejemplo, los siguientes n bloques libres.
- La última dirección de esa lista apunta a otro bloque que contiene el *siguiente* grupo de direcciones.

b. Ventajas y Desventajas:

- Ventajas:
  - Asignación masiva rápida: Si necesitas asignar muchos bloques de golpe, no tienes que recorrer la lista uno por uno. Lees solo el primer bloque y ya obtienes las direcciones de cientos de bloques libres instantáneamente.
- Desventajas:
  - Gestión un poco más compleja: Hay que gestionar bien los punteros cuando el bloque índice se llena o se vacía.

➢ **Recuento**

a. Descripción (Cómo trabaja): Es una técnica pensada para aprovechar la contigüidad. Generalmente, los bloques libres no están aislados, sino que suelen liberarse en "paquetes" (ej: liberaste un archivo que ocupaba 5 bloques seguidos).

- En lugar de mantener una lista de cada bloque individual (10, 11, 12, 13), guardamos parejas de datos:
  - [Dirección del primer bloque libre , Cantidad de bloques libres contiguos]
- *Ejemplo:* En lugar de guardar 100, 101, 102, 103, guardamos [100, 4].

b. Ventajas y Desventajas:

- Ventajas:
  - Lista más corta: Si el disco tiene muchos huecos grandes, la lista de espacio libre se hace pequeñísima, ahorrando espacio en disco o RAM.
  - Información de contigüidad: A diferencia de la lista enlazada normal, aquí el sistema sabe inmediatamente que hay un hueco de 4 bloques juntos, facilitando la *Asignación Contigua*.
- Desventajas:
  - Variable: Si el disco está muy fragmentado (muchos huecos de 1 solo bloque), la lista se vuelve igual de larga que una lista normal y perdemos la ventaja, teniendo que guardar además el contador (gastando más espacio).

37. Datos un disco con las características vistas en el ejercicio 29:

a. Calcule el tiempo de transferencia real de un archivo de 15 MiB(Mebibytes) almacenado en el disco de manera secuencial (todos sus bloques almacenados de manera consecutiva)

tiempo= 1.5 segundos aproximadamente

b. Calcule el tiempo de transferencia real de un archivo de 16 MiB(Mebibytes) almacenado en el disco de manera aleatoria.

Tiempo= 438 segundos aproximadamente

38. Unidad de Asignación. Las técnicas vistas pueden referenciar a sectores físicos de medio de almacenamiento o clusters (conjunto consecutivo de sectores físicos). Cite ventajas y desventajas de la utilización de cada forma de unidad de asignación.

1. Opción A: Usar Sectores Físicos (Unidad pequeña)

El Sector es la unidad mínima que viene de fábrica en el disco rígido (históricamente 512 bytes) . Si el Sistema de Archivos decide direccionar por sector, significa que para guardar un archivo, anota individualmente cada sector de 512 bytes que ocupa.

- Ventajas:
  - Menor Fragmentación Interna: Aprovechas el espacio al máximo. Si guardas un archivo de 600 bytes, usarás 2 sectores (1024 bytes) y solo desperdicias 424 bytes. Es ideal para guardar muchos archivos pequeñitos.

- Desventajas:
    - Tablas de Direccionamiento Gigantes: Si tienes un disco de 1 TB y anotas sectores de 512 bytes, necesitas una tabla con más de 2.000 millones de entradas. Como vimos en los slides, el tamaño de la tabla (como la FAT o mapa de bits) depende de la cantidad de bloques. Esto consume muchísima memoria RAM.
    - Lentitud (Overhead): El procesador tiene que gestionar millones de direcciones individuales.
    - Límite de tamaño de disco: Con arquitecturas viejas (como FAT16), si usas unidades muy chicas, se te acaban los números para direccionar y no puedes usar toda la capacidad del disco

2. Opción B: Usar Clusters (Conjunto de Sectores)

El Cluster (o Bloque lógico) es una agrupación de sectores consecutivos definida por el Sistema de Archivos (Software) . Por ejemplo, 1 Cluster = 8 Sectores (4 KB). El S.O. trata a esos 8 sectores como una única unidad indivisible.

- Ventajas:
    - Tablas de Direccionamiento Pequeñas: Al agrupar sectores, reduces drásticamente la cantidad de direcciones que el S.O. tiene que recordar. Una tabla FAT o de I-Nodos se vuelve mucho más chica y rápida de leer.
    - Lectura más rápida (Menos CPU): Leer 1 Cluster de golpe es más eficiente para el S.O. que pedir 8 veces un sector pequeño.
    - Permite Discos Grandes: Al usar unidades más grandes, puedes direccionar discos de mayor capacidad sin quedarte sin "números de dirección" (por ejemplo, FAT32 usa clusters de hasta 32KB para soportar discos grandes).
- Desventajas:
    - Mayor Fragmentación Interna: Este es el gran problema. Si tu Cluster es de 32 KB y guardas un archivo de texto de 1 KB (con la palabra "hola"), el archivo ocupará los 32 KB completos en el disco. Los 31 KB restantes están vacíos pero marcados como ocupados; nadie más los puede usar. Se desperdicia mucho espacio si tienes muchos archivos chicos.

39. Si se tiene un disco con 10 platos con 2 caras utilizables cada uno (20 cabezas), 350 cilindros y con 500 sectores por pista ¿Cuántos cluster pueden haber si se toma un tamaño de 4 sectores? ¿Y si el cluster es de 8 sectores?

Si es de 4 sectores el cluster entonces hay un total de (350*20*500)/4=875.000 clusters.

Si es de 8 sectores el cluster entonces hay un total de (350*20*500)/8=437.500 clusters.

40. Asumiendo un sistema de archivos que utiliza como unidad de asignación un cluster de 8 sectores de disco y que el primer cluster de datos (cluster 0) se comienza a almacenar en el sector LBA 120. Determine el los valores LBA de los sectores que corresponden a los

siguientes números de cluster:

LBA=Inicio+ (Nro de cluster * tamaño de cluster)

a. 0
LBA=120
b. 123
LBA=1104
c. 299
LBA=2512
d. 1001
LBA=8128

41. Describa la organización del FileSystem de Unix System V. Tenga en cuenta la organización del mismo, estructuras de datos, etc.

El sistema de archivos de UNIX System V divide cada partición de disco en cuatro áreas o bloques lógicos consecutivos:

1. Boot Block (Bloque de Arranque):
    - Se encuentra en el primer sector de la partición.

- Contiene el código necesario para iniciar (bootear) el Sistema Operativo (o el gestor de arranque).

2. Superblock (Superbloque):
    - Contiene los metadatos críticos y atributos globales del sistema de archivos.
    - Información que almacena: Tamaño total del sistema de archivos, cantidad de bloques libres, cantidad de I-Nodos libres y punteros a la lista de bloques libres.
3. I-Node List (Tabla de I-Nodos):
    - Es un área reservada que contiene una lista lineal de I-Nodos.
    - Definición de I-NODO: Es una estructura de control fundamental que representa a cada archivo (o directorio). Contiene toda la información administrativa del archivo excepto su nombre .
    - Datos en el I-Nodo: Permisos (lectura/escritura), ID del dueño, ID del grupo, tamaño del archivo, fechas (creación, acceso, modificación), contador de enlaces (links) y, lo más importante, los punteros a los Bloques de Datos donde está el contenido real.
4. Data Blocks (Bloques de Datos):
    - Es el área restante y más grande del disco.
    - Aquí se almacena el contenido real de los archivos y los directorios.

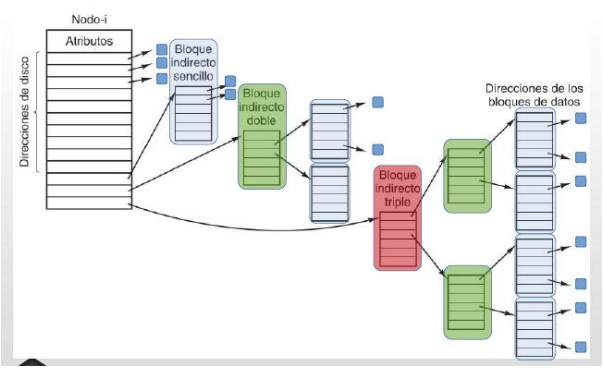
42. Gestión de archivos en *UNIX* System V

El sistema de archivos de UNIX utiliza una variante del esquema de asignación indexada para la administración de espacio de los archivos. Cada archivo o directorio está representado por una estructura que mantiene, entre otra información, las direcciones de los bloques que contienen los datos del archivo: el I-NODO.

Cada I-NODO contiene 13 campos para direcciones a los bloques de datos, organizadas de la siguiente manera:

- **10 de direccionamiento directo.**
- **1 de direccionamiento indirecto simple.**
- **1 de direccionamiento indirecto doble.**
- **1 de direccionamiento indirecto triple.**

a. Realice un gráfico que represente la estructura del *I-NODO* y de los bloques de datos. Cada cluster es de 1 Kib(Kibibits).



b. Si cada dirección para referenciar un bloque es de 32 bits:

i. ¿Cuántas referencias (direcciones) a bloque pueden contener un cluster?

Cantidad de direcciones= Tamaño Del cluster/ Tamaño de dirección

= 128 bytes/ 4 bytes= 32 direcciones se pueden tener.

ii. ¿Cuál sería el tamaño máximo de un archivo?

4.330.752 bytes

43. Dado el FileSystem de UNIX System V (visto en clase), indique qué cambios se producen el en mismo dada las siguientes situaciones:

a. Se agrega un nuevo archivo "Archivo1" en el directorio "Directorio1"

b. Se agrega nuevo contenido el archivo "Archivo1"

c. Se modifica el nombre del archivo "Archivo1" a "Archivo2"

d. Se hace un link duro a "Archivo2" en el mismo directorio con nombre "LinkArchivo2". e. Se borra un archivo "Archivo3".

a. Se agrega un nuevo archivo "Archivo1" en el directorio "Directorio1"

El sistema tiene que "nacer" el archivo y registrarlo.

1. Asignación de I-Nodo: Se busca un I-Nodo libre en el mapa de I-Nodos (o lista libre del Superbloque) y se marca como ocupado.
2. Inicialización del I-Nodo: Se llenan los datos en ese I-Nodo nuevo (Dueño, Permisos, Tamaño = 0, Fecha de creación).
3. Actualización del Directorio: Se va al bloque de datos de "Directorio1" y se agrega una nueva entrada (renglón):
   - Nombre: "Archivo1"
   - I-Nodo: [Número del I-Nodo asignado]
4. Metadatos del Directorio: Se actualiza el I-Nodo de "Directorio1" (cambia su fecha de modificación y quizás su tamaño si la tabla creció).

b. Se agrega nuevo contenido al archivo "Archivo1"

Ahora el archivo deja de estar vacío y necesita espacio real.

1. Verificación de Límites (Tu corrección): El Kernel calcula: *Tamaño Actual + Nuevos Datos = Nuevo Tamaño*. Luego se pregunta:

- ¿El nuevo tamaño supera el límite máximo teórico que soporta mi estructura de I-Nodo? (El límite que calculamos en el ejercicio 42).
- ¿El nuevo tamaño supera la cuota de disco asignada a este usuario?
- ¿Hay suficiente espacio libre en el disco para los nuevos bloques?
- *Si falla cualquiera de estas, la operación se cancela y te tira error (ej: "File too large" o "Disk quota exceeded").*

2. Asignación de Bloques: Se buscan bloques de datos libres en el mapa de bits (o lista libre del Superbloque) y se marcan como ocupados.
3. Escritura: Se escriben los datos del usuario en esos bloques físicos.
4. Actualización del I-Nodo: En el I-Nodo de "Archivo1":
    - Se actualizan los Punteros (Direcciones directas/indirectas) para que apunten a los nuevos bloques.
    - Se actualiza el Tamaño del Archivo (File Size).
    - Se actualiza la Fecha de Modificación.

c. Se modifica el nombre del archivo "Archivo1" a "Archivo2"

Esta es una "trampa" clásica: El archivo (I-Nodo) NO se entera de que cambió de nombre. El nombre es un concepto externo que vive en el directorio.

1. Lectura del Directorio: Se busca la entrada "Archivo1" en el bloque de datos de "Directorio1".
2. Modificación: Se sobrescribe la cadena de texto "Archivo1" por "Archivo2".
3. I-Nodo: El I-Nodo del archivo queda intacto (mismo número, mismos datos). Solo cambia el contenido del directorio que lo contiene.

d. Se hace un link duro a "Archivo2" en el mismo directorio con nombre "LinkArchivo2"

Un Link Duro es ponerle "otro nombre" al mismo perro (mismo I-Nodo).

1. Lectura del I-Nodo Original: Se averigua cuál es el número de I-Nodo de "Archivo2" (supongamos que es el #50).
2. Actualización del Directorio: Se agrega una nueva entrada en "Directorio1":
    - Nombre: "LinkArchivo2"
    - I-Nodo: #50 (El mismo número que el original).
3. Actualización del I-Nodo: Se va al I-Nodo #50 y se incrementa el campo Link Count (Contador de Enlaces) en +1. Ahora el archivo sabe que tiene 2 nombres apuntándolo.

e. Se borra un archivo "Archivo3"

Borrar en UNIX es "desvincular" (Unlink).

1. Busca y Elimina en Directorio: Se va al bloque de datos del directorio, se busca "Archivo3" y se elimina esa entrada (renglón).
2. Actualización del I-Nodo: Se va al I-Nodo correspondiente y se decrementa el Link Count en -1.
3. Decisión Final (¿Muere o vive?):
    - Si el *Link Count* todavía es mayor a 0 (tenía otros links duros), el archivo

sigue vivo y los datos no se tocan.

- Si el *Link Count* llega a 0 (era el último nombre):
  - Liberar Bloques: Se marcan todos los bloques de datos usados por el archivo como "Libres" en el Superbloque/Mapa de bits.
  - Liberar I-Nodo: Se marca el I-Nodo como "Libre".

44. Se tiene un FileSystem con un modelo indexado con niveles de indirección, donde se mantiene un i-nodo con 10 índices de direccionamiento directo, 1 índice para indirecto simple y 2 para indirecto doble. Considerando además que utilizan 32 bits para referenciar direcciones a bloques/clusters.

Suponga que existen dos siguientes archivos con sus tamaños logicos:

- ***ArchivoA*: de 55 KibiBytes**
- ***ArchivoB*: de 12 KibiBytes**
- ***ArchivoC*: de 1075 KibiBytes**

a. Si se utiliza como unidad de asignación el sector de disco de 512bytes

i. ¿Cuántas direcciones a sectores puede contener un sector de disco?

128 direcciones.

ii. ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?

El tamaño máximo de un archivo seria de 16.847.872 .

iii. ¿Cuántos sectores se necesitan para almacenar cada uno de los archivos antes enumerados? Indique el tamaño físico que se requirió para

almacenar cada uno.

Para el archivo de 55 kibibytes necesitamos 110 sectores.
Tamaño físico en bloques= 110 bloques + 1bloque(ya que se necesita 1 indice para indirecto simple)= 111 bloques en total.
Tamaño físico en bytes = 111*512=56.832 bytes.

Para el archivo de 12 kibibytes necesitamos 24 sectores.
Tamaño físico en bloques= 24 bloques + 1bloque(ya que se necesita 1 indice para indirecto simple)= 25 bloques en total.
Tamaño físico en bytes = 25*512=12.800 bytes.

Para el archivo de 1075 kibibytes necesitamos 2.150 sectores.
Tamaño físico en bloques= 2150 bloques + 1bloque(ya que se necesita 1 indice para indirecto simple) + 17 = 2168 bloques en total.
Tamaño físico en bytes = 2168*512=1.110.016 bytes.

iv. Indique la fragmentación interna que se produce para cada uno de los archivos antes enumerados.

No hay fragmentación interna para ninguno de los 3 archivos.

b. Si se utiliza como unidad de asignación el cluster de disco de 2 KibiBytes.

i. ¿Cuántas direcciones a clusters puede contener un cluster de disco?

direcciones= 512

ii. ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?

1.049.620kiB

iii. ¿Cuántos clusters se necesitan para almacenar cada uno de los archivos antes enumerados? Indique el tamaño físico que se requirió para almacenar cada uno.

Para el archivo de 55 kibibytes necesitamos 27,5 sectores.
Tamaño físico en bloques= 28 bloques + 1bloque(ya que se necesita 1 indice para indirecto simple)= 29 bloques en total.
Tamaño físico en bytes = 29*2kiB=58kiB
fragmentación interna= 1kiB

Para el archivo de 12 kibibytes necesitamos 6 sectores.
Tamaño físico en bloques= 6 bloques en total.
Tamaño físico en bytes = 6*2kiB=12 kiB.
Fragmentación interna=0

Para el archivo de 1075 kibibytes necesitamos 537.5 sectores.
Tamaño físico en bloques= 538 bloques + 1bloque(ya que se necesita 1 indice para indirecto simple) + 2 = 541 bloques en total.
Tamaño físico en bytes = 541*2kiB=1082kiB.
Fragmentación interna=1kiB

iv. Indique la fragmentación interna que se produce para cada uno de los archivos antes enumerados.

45. Para los incisos a. y b. el ejercicio anterior, y considerando que el fileSystem se utiliza en un disco con las siguientes características:

- **➢ 6 platos, con 2 caras útiles cada uno**
- **➢ 2500 pistas por cara y 100 sectores por pista de 512 bytes cada uno. ➢ 7200 RPM**
- **➢ seek type de 10,5 ms**
- **➢ velocidad de transferencia de 146 MiB/seg (Mebibytes por segundo)**

Calcule el tiempo de transferencia de los 3 archivos (ArchivoA, ArchivoB y ArchivoC) del ejercicio previo.

Inciso a)

- ArchivoA: de 55 KibiBytes

111 bloques en total entonces:

111*(4,16+ 10,5+ 0,0034)=1,6 segundos

- ArchivoB: de 12 KibiBytes

  24 bloques en total

  24 bloques en total entonces:

  24*(4,16+ 10,5+ 0,0034)=0,35 segundos

- ArchivoC: de 1075 KibiBytes

  2150 bloques en total

  2150*(4,16 + 10,5 + 0,0034)=31 segundos

Inciso b)

Tiempo de tranferencia por cluster= 0,013 segundos

Tiempo total por cluster= (0.013 + 10,5 + 4,16) =14,67

- ArchivoA: de 55 KibiBytes

  28 bloques

  14,67*28=0,4 segundos

- ArchivoB: de 12 KibiBytes

  6 bloques

  14,67*6=0,08 segundos

- ArchivoC: de 1075 KibiBytes

  538 bloques

  14,67*538=7,8 segundos

46. Analice las siguientes fórmulas necesarias para localizar un I-NODO en la tabla de i-nodos de un filesystem similar a System V (visto en la teoría)

<u>nro bloque = ((nro de inodo -1)/nro. de inodos por bloque) + bloque de comienzo de la lista de inodos.</u>

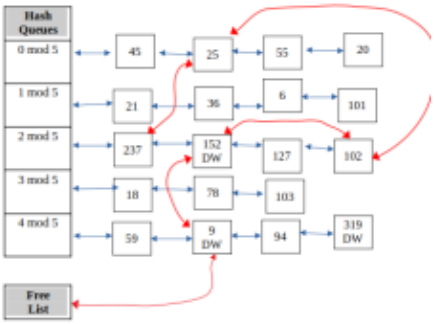
Desplazamiento del inodo en el bloque = ((nro de inodo - 1) módulo (número de inodos por bloque)) * medida de inodo del disco.

a. Según la primera fórmula, asumiendo que en el bloque 2 está en el comienzo de la lista de inodos y que hay 8 inodos por bloque: calcule donde se encuentra el inodo 8 y el 9. ¿Dónde estarían para un bloque de disco con 16 inodos?

b. De acuerdo a la segunda fórmula, si cada inodo del disco ocupa 64 bytes y hay 8 inodos por bloque de disco, el inodo 8 comienza en el desplazamiento 448 del bloque de disco. ¿Dónde empieza el 6? Si fueran inodos de 128 bytes y 24 inodos por bloque: ¿dónde empezaría el inodo 8?

Caché de Disco

47. Dado el siguiente gráfico que representa el estado de Buffer Cache en Unix System V (Visto en clase):



Responda a los siguientes incisos (considere cambios en los headers, las hash queue, free list, operaciones de E/S, etc):

a. ¿Qué sucedería si un proceso P1 requiere el bloque 45?
b. ¿Qué sucedería si un proceso P1 solicita el bloque 113?
c. ¿Qué sucedería si un proceso P1 solicita el bloque 102?
d. ¿Qué sucedería si un proceso P1 solicita el bloque 9?